



NUST
NATIONAL UNIVERSITY
OF SCIENCES & TECHNOLOGY

EDGE-DEPLOYED 3D VISION AND LOCALIZATION FOR AUTONOMOUS SYSTEMS

Final Year Project

Submitted By

Rayyan Naeem - 414044

Maleeha - 414469

In Partial Fulfillment

Of the Requirements for the degree

Bachelor of Electrical Engineering (BEE)

School of Electrical Engineering and Computer Science

National University of Sciences and Technology

Islamabad, Pakistan

April 30, 2026

DECLARATION

We hereby declare that this project report entitled “EDGE-DEPLOYED 3D VISION AND LOCALIZATION FOR AUTONOMOUS SYSTEMS” submitted to the School of Electrical Engineering and Computer Science (SEECs), is a record of an original work done by us under the guidance of [Supervisor Dr. Jameel Nawaz](#) and [Co-Supervisor Dr. Wajid Mumtaz](#). We further affirm that all sources used in this report have been appropriately acknowledged with citations, and no part has been plagiarized. This project report has been submitted to fulfill the Bachelor of Electrical Engineering degree requirements.

Team Members

Signature

Rayyan Naeem

Maleeha

Supervisor's Name

Dr. Jameel Nawaz

Dr. Wajid Mumtaz

ABSTRACT

This project report presents the complete design and implementation of a real-time, metric-scale monocular Visual Simultaneous Localization and Mapping (VSLAM) system for a differential-drive robot powered by a Raspberry Pi 4B microcomputer. Indoor environments deny GPS access; low-cost mobile robots therefore require self-contained localization that operates within severe computational budgets.

The system employs a hybrid Lucas-Kanade optical-flow and ORB feature-matching frontend that sustains 10 Hz pose estimation on the Pi without a GPU. An asynchronous mapping thread, combined with a custom sparse Jacobian formulation for Bundle Adjustment, achieves a $50\times$ speedup over a naive dense implementation—reducing per-keyframe optimization from 4,000ms to under 120ms—enabling the mapping backend to run concurrently without blocking live tracking.

A finite-state machine with hysteresis governs tracking confidence, and a novel breadcrumb backtracking mechanism uses wheel odometry to physically reverse the robot into the mapped region following complete localization failure. Global drift is corrected by a Bag-of-Words loop-closure detector with geometric PnP-RANSAC verification, followed by 3-DoF Pose Graph Optimization, reducing 100 m trajectory error from 1.6 m to 0.18 m. Algorithmic validation and rapid prototyping were accelerated using a custom ROS 1 and Gazebo simulation pipeline prior to physical edge deployment.

Keywords: Visual SLAM, Monocular Localization, Edge Computing, Raspberry Pi, Bundle Adjustment, Loop Closure, Autonomous Navigation, Wheel Odometry Fusion.

ACKNOWLEDGEMENTS

First and foremost, we are profoundly grateful to Allah Almighty for granting us the strength, patience, and perseverance to complete this project.

We would like to express our sincere gratitude to our supervisor, Dr. Jameel Nawaz, and co-supervisor, Dr. Wajid Mumtaz, for their invaluable guidance, support, and continuous encouragement throughout this project. Their expertise and constructive feedback were instrumental in shaping the direction of this work. Along with the above-mentioned personalities, our parents deserve credit for believing in us and investing their time, energy, and monetary resources in us.

Table of Contents

<i>April 30, 2026</i>	1
<i>DECLARATION</i>	2
<i>Team Members Signature</i>	2
<i>Supervisor's Name</i>	2
ABSTRACT	3
<i>LIST OF FIGURES</i>	6
<i>LIST OF TABLES</i>	7
1: INTRODUCTION	8
1.1 Background and Motivation	8
1.2 Problem Statement	8
1.3 Proposed Solution	9
1.4 Report Outline	10
2: LITERATURE REVIEW	10
2.1 Foundations of Visual SLAM.....	10
2.2 Parallel Tracking and Mapping (PTAM)	11
2.3 ORB-SLAM	11
2.4 Feature Extraction and Matching	11
2.5 PnP Pose Estimation.....	11

2.6	Bundle Adjustment.....	12
2.7	Place Recognition and Loop Closure	12
2.8	Visual Odometry and Scale Recovery	12
2.9	Human Pose Estimation on Edge Devices	13
3.	PROBLEM DEFINITION	14
3.1	The Need for Autonomous Indoor Localization	14
3.2	Significance of the Problem	14
3.3	Target Users and Impact.....	15
4.	SYSTEM ARCHITECTURE AND DESIGN.....	16
4.1	High-Level System Overview	16
4.1	Hardware Platform	17
4.2	Software Thread Architecture.....	18
4.3	Simulation Architecture.....	19
4.4	Human Pose Estimation Subsystem (Auxiliary Operator Perception).....	19
5.	DETAILED PIPELINE AND ALGORITHMS	20
5.1	Stage 0: Offline Camera Calibration.....	20
5.2	Stage 1: Map Initialization.....	20
5.3	Stage 2: Live Tracking Frontend.....	23
5.4	Stage 3: Keyframe and Map Management	26
5.5	Stage 4: Tracking State Machine.....	28
	<i>States:</i>	28
	<i>Transitions (with Hysteresis):</i>	29
5.6	Stage 5: Local Bundle Adjustment.....	29
5.7	Stage 6: Loop Closure Detection and Correction	31
5.8	Stage 7: Wheel Odometry Integration	33
5.9	Stage 8: Autonomous Navigation.....	34
	<i>State Machine:</i>	34
	<i>Safety Gates (The "Swiss Cheese" Model):</i>	35
5.10	Stage 9: Human Pose Estimation and Scale-Free Depth.....	35
6.	MAJOR CHALLENGES AND SOLUTIONS.....	38

6.1	Software Challenges	38
6.2	Hardware and Systems Integration Challenges	42
6.3	Hardware Iteration Bottleneck.....	45
7.	IMPLEMENTATION AND TESTING	46
7.1	Implementation Details	46
7.2	Testing Methodology and Metrics	47
7.3	Simulation as a Prototyping Tool	47
7.4	Human Pose Estimation and Scale-Free Depth Inference	48
7.5	Algorithmic Benchmarking: Our Implementations vs Standard Alternatives	50
8.	RESULTS AND DISCUSSION	56
8.1	Initialization and Scale Recovery Performance	56
8.2	Live Tracking Speed and LK/ORB Hybrid Transitions.....	58
8.3	Local Bundle Adjustment and Map Churn Mitigation.....	60
8.4	Loop Closure Performance.....	61
8.5	Navigation and Waypoint Accuracy.....	62
8.6	COMPARATIVE ANALYSIS: OUR SYSTEM VS PUBLISHED MONOCULAR SLAM.....	63
8.7	Failure Recovery Performance.....	66
8.8	Human Pose Estimation Performance and Telemetry	66
9.	CONCLUSION AND FUTURE WORK	68
9.1	Conclusion.....	68
9.2	Future Work	69
	REFERENCES.....	71

LIST OF FIGURES

Figure 1: System Architecture Diagram (Hardware + Software Dataflow)

Figure 2: Raspberry Pi Robot Hardware Platform and Component Layout

Figure 3: Analysis of 35 NPZ initial map files generated during offline initialization.

Figure 4(a,b,c): Direct latency comparison between Lucas-Kanade optical flow (primary tracker) and ORB feature matching (fallback tracker).

Figure 5: Distribution of PnP inlier counts comparing LK tracking (median ~171 inliers) vs. ORB fallback (median ~39 inliers).

Figure 6: Algorithm benchmark summary — speedup factors at representative scales. Values $>1\times$ indicate our implementation is faster.

Figure 7: PnP RANSAC solve time across correspondence counts.

Figure 8: Jacobian matrix construction time scaling with problem size.

Figure 9: Local BA solve time across configurations (3 KFs, 120 pts).

Figure 10: PGO solve time: SE(2) 3-DOF vs SE(3) 6-DOF.

Figure 11: BoW scoring latency vs vocabulary size (200 candidates).

Figure 12: Frontend latency scaling with tracked point count.

Figure 13: Pruning time scaling with map size (log-log).

Figure 14: Map growth dynamics.

Figure 15: Comprehensive performance dashboard summarizing 275 experimental run.

Figure 16: Local Bundle Adjustment performance analysis across 405 BA events.

Figure 17: Per-component median runtime comparison. Numbers for ORB-SLAM2/3, DSO, and SVO2 are from their respective publications running on desktop i7 CPUs. Our measurements are from Raspberry Pi 4B (ARM Cortex-A72 @ 1.8 GHz).

Figure 18: System capability comparison across nine functional dimensions. Full support (✓), partial/in-progress (●), not available (X). Our row is highlighted in blue.

Figure 19: Self-derived tracking quality from live Pi 4B runs (122,876 frames, 3,715 keyframe events).

LIST OF TABLES

Table 1: Quantitative comparison of LK tracking path versus ORB fallback path using real robot run data. LK achieves 4-5x faster frame processing (17.4ms median) with higher inlier counts (171 median) while maintaining 85% tracking share.

Table 2: Qualitative algorithm comparison highlighting key architectural decisions distinguishing our edge-deployed implementation from standard academic approaches, with measured advantages specific to Raspberry Pi 4B deployment

1: INTRODUCTION

1.1 Background and Motivation

The proliferation of mobile robots in logistics, inspection, and domestic service has created urgent demand for robust, low-cost localization. High-performance systems rely on 3D LiDAR sensors costing thousands of dollars; the pursuit of affordable autonomy instead demands that cheap sensors carry full navigational responsibility. A monocular RGB camera—available for under PKR 6,000—is the most ubiquitous visual sensor, yet extracting reliable 3D pose from its 2D image stream is a fundamentally ill-posed problem.

Indoor environments simultaneously deny access to the Global Navigation Satellite System (GNSS) and present challenges absent in open-air settings: featureless white walls, repetitive corridor textures, and narrow fields of view. Robots must therefore rely entirely on proprioceptive data (wheel encoders) and exteroceptive perception (camera) for self-localization. Simultaneous Localization and Mapping (SLAM) has emerged as the standard solution, enabling a robot to build an environment map while tracking its pose within that map—all in real time.

This work targets the intersection of three demanding constraints: monocular input (no stereo baseline, no depth sensor), metric accuracy (the map must carry physical scale for navigation), and edge deployment (a Raspberry Pi 4B with four ARM Cortex-A72 cores at 1.8 GHz and 8 GB LPDDR4 RAM, without a discrete GPU). No existing open-source SLAM system simultaneously satisfies all three constraints out of the box.

The system is inspired directly by the architecture of PTAM (Klein & Murray, 2007) and ORB-SLAM (Mur-Artal et al., 2015), the two landmark systems that defined modern feature-based visual SLAM, but re-implemented from scratch in Python/OpenCV for our specific hardware constraints.

1.2 Problem Statement

Five interrelated problems block deployment of monocular VSLAM on a Raspberry Pi for practical indoor navigation:

1. **Computational Bottleneck:** Bundle Adjustment—the gold-standard map refinement step requires 4–5 seconds per keyframe on the Pi using standard dense solvers, blocking the tracking thread and causing immediate localization failure.

2. **Monocular Scale Ambiguity:** A single camera cannot recover the absolute metric scale of translation. Without an external reference, the reconstructed map scales arbitrarily, making distance-based navigation impossible.
3. **Tracking Fragility:** Fast rotations, motion blur from a rolling-shutter camera, and featureless surfaces cause feature trackers to fail. Most academic systems provide no systematic recovery mechanism.
4. **Hardware Imperfections:** Low-cost motors exhibit a large deadband and nonlinear response; differential-drive skid during turns introduces severe odometry drift; the camera's default exposure settings produce blurred images during motion.
5. **System Integration Gap:** Academic papers focus on individual algorithmic components and rarely address the engineering required to integrate SLAM, motor control, serial communication, and navigation into one coherent, real-time system.

1.3 Proposed Solution

We address all five problems through a fully integrated, asynchronous pipeline with the following core contributions:

1. **Hybrid LK+ORB Frontend:** Lucas-Kanade optical flow handles fast, low-latency tracking under normal conditions; ORB feature matching provides robust re-localization when tracking confidence falls.
2. **Sparse Jacobian Bundle Adjustment:** A custom sparsity pattern passed to SciPy's trust-region solver reduces BA time from 4000+ms to <500ms, enabling asynchronous background execution.
3. **State-Machine with Hysteresis and Backtracking:** A three-state FSM (GOOD / WEAK / RELOCALIZING) with hysteresis prevents mode thrashing; a breadcrumb backtracker physically reverses the robot along its stored odometry trail to re-enter the mapped region.
4. **Sensor Fusion for Scale:** Wheel odometry provides a PnP initial guess, a motion-prior sanity gate, and an absolute scale multiplier for the navigation layer.
5. **Hardware Mitigation:** Camera exposure profiles suppress blur; deadband compensation and EMA-based bias correction linearize the odometry model.
6. **Zero-Modification Simulation Bridge:** A Python ``sys.modules`` injection architecture

that intercepts hardware calls, allowing the exact edge-Python SLAM codebase to be tested within a ROS/Gazebo virtual environment without modifying a single line of core logic.

1.4 Report Outline

Chapter 2 surveys foundational SLAM literature. Chapter 3 formally defines the problem and its societal context. Chapter 4 presents the hardware platform and high-level software architecture. Chapter 5 details every algorithmic stage of the pipeline. Chapter 6 documents major challenges and the specific engineering solutions developed. Chapter 7 describes implementation details and testing methodology. Chapter 8 presents and discusses experimental results. Chapter 9 concludes with future research directions.

2: LITERATURE REVIEW

This chapter reviews the foundational algorithms and seminal systems that form the theoretical and architectural backbone of our work. We trace the evolution of visual SLAM from early filtering-based approaches to modern graph-optimization frameworks, with particular emphasis on PTAM's parallel threading paradigm and ORB-SLAM's comprehensive feature-based pipeline. The chapter also examines critical supporting techniques such as feature detection, optical flow, PnP pose estimation, Bundle Adjustment, loop closure, and visual odometry scale recovery that are subsequently reimplemented and optimized for our edge-deployed platform.

2.1 Foundations of Visual SLAM

The mathematical underpinning of all feature-based SLAM is multi-view geometry, comprehensively treated by Hartley and Zisserman (2003). Their central result—that structure and motion can be recovered from image correspondences only up to an unknown scale factor under a monocular camera model—defines the fundamental ambiguity this thesis must resolve. The pinhole camera model, projection equations, essential and fundamental matrices, and the reprojection error formulation used throughout our pipeline derive directly from this reference.

The goal of visual SLAM is to estimate the trajectory of a camera (the 6-Degrees-of-Freedom (DoF) pose) and build a map of the environment using only visual input. This is typically achieved by extracting distinctive features (e.g., corners, blobs) from images, matching them across frames, and using geometric principles to triangulate their 3D positions.

Early work by Davison et al. on MonoSLAM (2007) demonstrated that a single camera could localize a hand-held device in real time using an Extended Kalman Filter (EKF). While EKF-based approaches offer elegant probabilistic formulations, they scale poorly ($O(n^2)$ state updates) and produce inferior accuracy compared to later graph-based methods. Nevertheless, MonoSLAM established the field's vocabulary and proved monocular SLAM's feasibility.

2.2 Parallel Tracking and Mapping (PTAM)

Klein and Murray's PTAM (2007) was a watershed contribution that directly inspired our architecture. Its central innovation—decoupling tracking and mapping into two parallel threads—allowed real-time 30 Hz tracking while a heavier Bundle Adjustment ran asynchronously on keyframes in the background. This separation is the structural backbone of our system. PTAM also introduced the concept of keyframe-based mapping: instead of incorporating every frame, only frames with sufficient baseline from the previous keyframe are inserted, controlling map growth.

2.3 ORB-SLAM

Mur-Artal, Montiel, and Tardós introduced ORB-SLAM (2015), extending PTAM's two-thread design to a three-thread architecture by adding a dedicated Loop Closing thread. Their choice of ORB (Oriented FAST and Rotated BRIEF) features (Rublee et al., 2011) patent-free, binary-descriptor, and computable in milliseconds makes the system practical on modest hardware. We directly adopt their map point culling heuristic (found/visible ratio thresholding) and their geometric verification approach for loop closure candidates. ORB-SLAM2 (Mur-Artal & Tardós, 2017) further extended the framework to stereo and RGB-D cameras, but the monocular branch remains the reference for our work.

2.4 Feature Extraction and Matching

ORB features combine FAST corner detection for speed (Rosten & Drummond, 2006) with a binary BRIEF descriptor (Calonder et al., 2010), achieving rotation invariance via a learned orientation. Matching is performed via Hamming distance—a single XOR operation per 32-byte block—enabling thousands of comparisons per millisecond. Lowe's ratio test (Lowe, 2004), which accepts a match only if the best-to-second-best distance ratio is below 0.75, provides the primary ambiguity filter for our ORB fallback tracker.

Lucas and Kanade's optical flow formulation (1981), extended to image pyramids by Bouguet (2001), provides the primary tracking mechanism. By assuming brightness constancy within a local patch and solving iteratively on a Gaussian pyramid, LK flow tracks existing correspondences at under 0.1ms per point—orders of magnitude faster than re-detection and description.

2.5 PnP Pose Estimation

The Perspective-n-Point (PnP) problem estimating camera pose from 3D-to-2D correspondences—is central to live tracking. Lepetit, Moreno-Noguer, and Fua's EPnP algorithm (2009) solves the problem in $O(n)$ time by expressing 3D points as weighted sums of four control points, yielding a closed-form algebraic solution without iterative refinement. EPnP wrapped in RANSAC (Fischler & Bolles, 1981) provides our primary pose estimator, capable of handling the

40–60% outlier rates typical of monocular SLAM tracking.

2.6 Bundle Adjustment

Bundle Adjustment (BA), as synthesized by Triggs et al. (2000), jointly minimizes reprojection error over a window of camera poses and 3D points using nonlinear least squares. The Jacobian structure in BA is inherently sparse—each observation connects exactly one pose and one point—enabling specialized solvers (Schur complement, sparse Cholesky) to achieve dramatic speedups over dense methods. We exploit this sparsity through SciPy's `'least_squares'` interface with an explicit sparsity pattern, achieving real-time performance without writing native C++ code.

2.7 Place Recognition and Loop Closure

Gálvez-López and Tardós introduced DBoW2 (2012), a Bag-of-Words framework using binary ORB descriptors and a pre-trained hierarchical vocabulary. Each keyframe is represented as a normalized histogram of visual word frequencies, enabling $O(\log N)$ database queries for loop candidate retrieval. Histogram intersection scoring provides a fast similarity measure. We implement an equivalent BoW pipeline directly in NumPy, preserving the algorithmic structure of DBoW2 without its C++ dependency.

Pose Graph Optimization (PGO), as formalized in the g2o framework by Kümmerle et al. (2011), corrects drift by treating the trajectory as a graph whose edges represent relative pose constraints. A loop closure adds a long-range edge; PGO minimizes the total residual error over all edges. Our 3-DoF ground-plane simplification (x, z, yaw) reduces the optimization from 6-DoF $SE(3)$ to a 2D rigid body problem, enabling fast Python-based solving without a specialized graph-optimization library.

2.8 Visual Odometry and Scale Recovery

Scaramuzza and Fraundorfer's comprehensive tutorial (2011) identifies monocular scale recovery as the critical unsolved problem in single-camera VO. Common approaches include temporal scale propagation (scaling each translation by the previous frame's metric ratio), ground-plane-constrained geometry, and external sensor fusion. We adopt the latter, fusing wheel odometry as a metric scale source—an approach validated by the authors as the most reliable for ground vehicles operating on planar surfaces.

Key distinction from all prior work: Every component has been reimplemented from scratch in Python/OpenCV to run within the memory and CPU constraints of

a Raspberry Pi 4B, with all real-time decisions (collision avoidance, motor safety) delegated to a dedicated Arduino microcontroller, ensuring the Pi's SLAM pipeline never blocks on hardware safety logic.

2.9 Human Pose Estimation on Edge Devices

While SLAM focuses on geometric structure recovery, real-world robots benefit from concurrent human presence awareness for operator interaction and safety. MediaPipe Pose (Lugaresi et al., 2019) is a single-stage pose detector that runs efficiently on CPU-only edge devices by combining a lightweight CNN feature extractor with blazepose, a mobile-optimized skeleton model. Unlike two-stage methods (e.g., Faster R-CNN + OpenPose), single-stage detectors sacrifice per-frame latency for deployability on constrained hardware.

For scale estimation, monocular human pose does not directly provide metric depth; approaches include temporal scale propagation, ground-plane assumptions, and anthropometric priors. We adopt the latter: a set of fixed human-body measurements (shoulder width, hip width, height) act as scale references. This approach is distinct from full 3D human mesh recovery (e.g., SMPL-based methods) and prioritizes speed and simplicity over anatomical accuracy.

Video-mode pose estimation (temporal consistency across frames) offers robustness advantages over per-frame IMAGE mode by leveraging frame-to-frame coherence, at the cost of introducing a small processing delay. For operator feedback on robots, this trade-off is acceptable.

3. PROBLEM DEFINITION

3.1 The Need for Autonomous Indoor Localization

Indoor environments—offices, laboratories, warehouses, hospitals—represent the primary deployment domain for small mobile service robots. Tasks such as inventory scanning, mail delivery, environmental monitoring, and automated inspection all require reliable autonomous navigation. GPS signals are attenuated by structural materials to the point of unusability indoors, making satellite-based localization unavailable. Wheel odometry alone accumulates unbounded drift; without correction, a robot's estimated position diverges from reality within minutes.

The economic case for low-cost solutions is compelling. Industrial Automated Guided Vehicles (AGVs) equipped with 3D LiDAR sensors cost PKR 2,800,000–28,000,000. A Raspberry Pi 4B (PKR 43,000) paired with a USB camera (PKR 7,000) and an Arduino UNO (PKR 11,000) totals under PKR 60,000–65,000. If a viable SLAM system can operate on this platform, it democratizes autonomous navigation for small businesses, university laboratories, and independent researchers.

Problem statement: Indoor environments lack GPS. Small robots need low-cost, reliable localization. Affordable autonomy for Labs and delivery.

Our Solution: Raspberry Pi + Arduino Platform, Visual Odometry + Wheel Odometry Fusion, CV + Robotics Integration.

3.2 Significance of the Problem

The challenges that make this problem non-trivial fall into five categories:

3.2.1 Scale Ambiguity

Monocular geometry recovers translation direction but not magnitude. Formally, given two frames with matched pixel correspondences, the Essential Matrix E decomposes into rotation R and a unit translation vector $\hat{t}$. The actual translation is $\lambda\hat{t}$ for some unknown scalar λ . Without metric scale, navigation commands ("move 1.5 m forward") cannot be generated from visual pose alone.

3.2.2 Low-Texture and Repetitive Environments

ORB features require intensity gradients; blank walls, uniformly carpeted floors, and identically painted doors provide few or none. In these regions, triangulation is impossible and tracking fails. Many real office environments contain extended low-texture zones between feature-rich areas such as desks and bookshelves.

3.2.3 Real-Time Constraints on Edge Hardware

The Raspberry Pi 4B provides approximately 10–15 GFLOPS of single-precision compute across four cores. Running ORB detection on a 640×480 image takes approximately 80–150ms; Bundle Adjustment on 5 keyframes naively takes 4,000ms. Threading these operations naively produces a system that processes at sub-1 Hz—orders of magnitude below the 10 Hz minimum for smooth navigation.

3.2.4 Map Resource Explosion

Each keyframe contributes 10–50 new 3D points, each carrying a 32-byte ORB descriptor. Over a 30-minute run with keyframes every 2 s, the map accumulates up to 27,000 points. Nearest-neighbor descriptor matching— $O(N)$ in map size becomes the dominant latency contributor, eventually exceeding 1 s per query.

3.2.5 Hardware Imperfections

Rolling-shutter cameras produce skewed images during rotation, violating the instantaneous-capture assumption of all feature-based methods. Low-cost DC motors exhibit deadband (zero output below a threshold PWM), velocity nonlinearity, and stiction. Differential-drive robots on carpet experience significant lateral wheel slip during turns, making simple encoder-based odometry unreliable beyond 2–3m.

3.3 Target Users and Impact

Three communities stand to benefit from a validated, open-source low-cost VSLAM system:

1. **Academic Researchers:** Robotics and computer vision groups needing a low-cost platform for teaching SLAM concepts and prototyping novel algorithms without expensive sensor investment.
2. **Hobbyists and Makers:** The open-source community building home-mapping robots, automated lawnmowers, or warehouse helpers on commodity hardware.
3. **Small-Scale Industry:** Businesses needing automated inspection or inventory tracking in small facilities where deploying a PKR 165,178 LiDAR AGV is economically unviable.

A working system meeting the target specification—10 Hz tracking, <0.15 m waypoint error at 3m, autonomous loop-closure-corrected drift below 0.2 m per 100 m would directly enable practical autonomous navigation for all three groups at a hardware cost below PKR 70,000.

4. SYSTEM ARCHITECTURE AND DESIGN

4.1 High-Level System Overview

The system is architected around two computing nodes and an event-driven multithreaded architecture, as elaborated in [4.3 Software Thread Architecture](#). The Raspberry Pi 4B executes all SLAM computation, state estimation, and navigation logic in Python/OpenCV. The Arduino Uno handles all real-time, safety-critical hardware interactions—motor PWM output, command-velocity dead reckoning, ultrasonic obstacle detection—entirely independently of the Pi, ensuring that hardware collision avoidance functions even if the Pi's SLAM pipeline stalls.

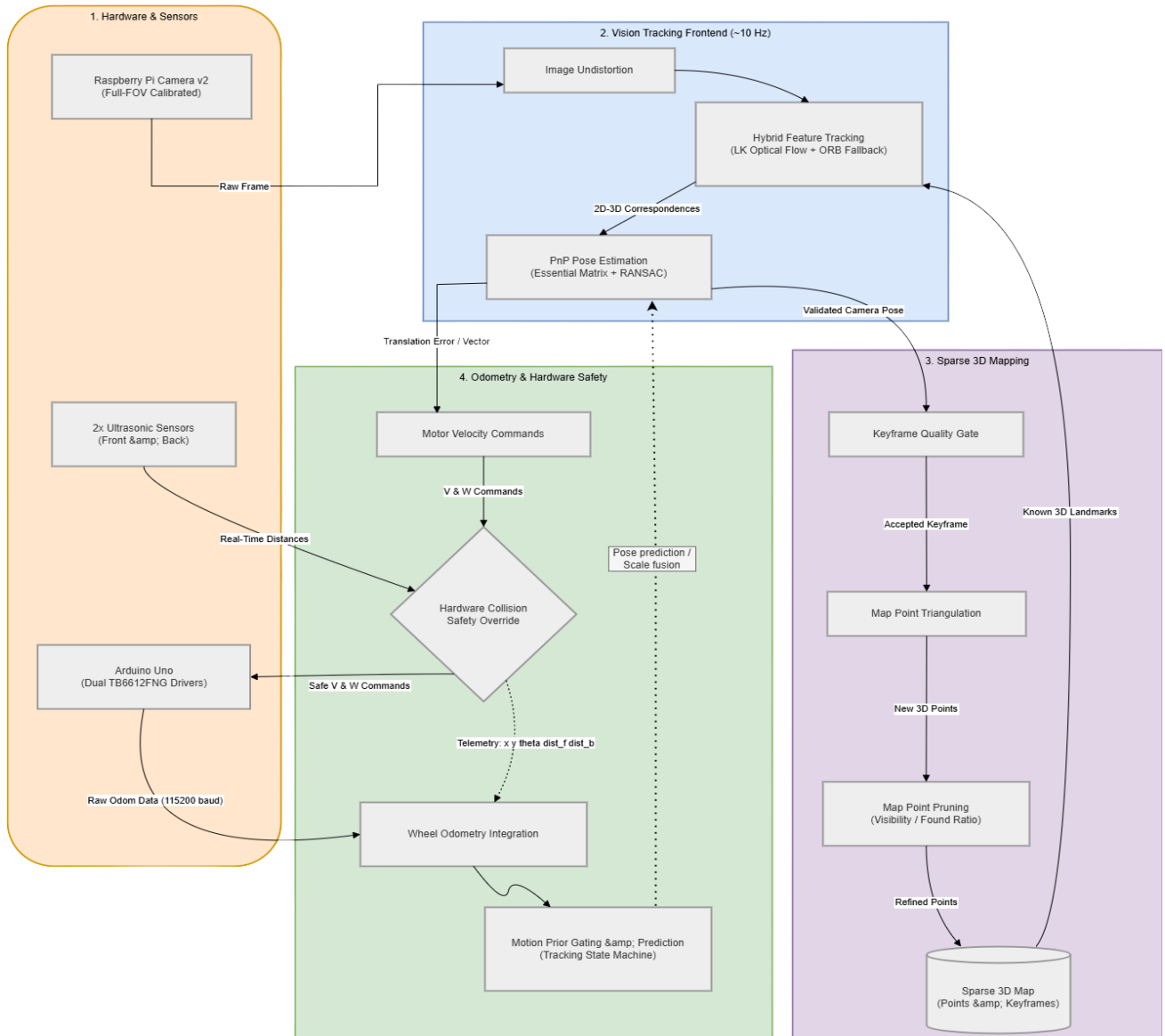


Figure 1. Full system architecture diagram.

The two nodes communicate via a 115,200-baud serial link. The Pi transmits linear velocity commands (v_cmd) and angular velocity commands (w_cmd) in SI units. The Arduino streams back raw odometry increments ($ODOM \Delta x \Delta y \Delta \theta$) at 20 Hz, via open-loop PWM estimation. This one-way safety architecture guarantees that the Arduino's collision avoidance firmware can override and stop the motors regardless of Pi-side computation state.

4.1 Hardware Platform



Figure 2. Raspberry Pi robot hardware platform and component layout.

The robot platform is a custom differential-drive chassis approximately 30×30 cm in footprint. Key components are:

COMPONENT	SPECIFICATION	ROLE
<i>Raspberry Pi 4b</i>	4 GB LPDDR4, 4× ARM Cortex-A72 @1.8 GHz, Ubuntu Server 22.04	Computation
<i>Arduino uno</i>	C++ firmware for real-time motor and sensor control	Microcontroller
<i>3-cell LiPo battery</i>	separate 5 V regulated rail for Pi, direct 9 V for motors	Power
<i>Raspberry Pi Camera Module v2</i>	8 MP Sony IMX219, rolling shutter, 640×480 @ 30 Hz	Camera
<i>TB6612FNG dual H-bridge drivers</i>	Dual DC motors	Drive system
<i>Ultrasonic sensors</i>	2× HC-SR04 ultrasonic sensors (front and rear), polled entirely by Arduino	Obstacle detection

4.2 Software Thread Architecture

The Pi-side software comprises an asynchronous Python architecture, designed around a producer-consumer pattern to maximize CPU utilization and minimize inter-thread blocking:

1. **Camera Capture Thread (Asynchronous):** Grabs frames via Picamera2, applies a pre-computed undistortion remap, timestamps each frame, and writes to a shared frame buffer. Short exposure (12,000 μ s/12ms maximum) is enforced via camera driver controls to minimize rolling-shutter blur during motion.
2. **Main Tracking & Navigation Loop (~10 Hz):** Reads the latest undistorted frame, executes the hybrid LK/ORB frontend, and estimates the pose. Crucially, it leverages the Arduino's odometry stream to seed an Iterative Levenberg-Marquardt PnP solver for hyper-fast convergence during smooth motion, falling back to algebraic EPnP-RANSAC only when the odometry prior fails. It then validates the pose, updates the system state machine, and synchronously executes the 'WaypointNavigator' to issue velocity commands to the serial bridge.
3. **Mapping Thread (Async Background):** Reads keyframes from a single-slot non-blocking queue. When a keyframe is available, performs triangulation, local Bundle Adjustment, and map point pruning, then triggers the loop closer. If the queue is full when a new keyframe arrives, the new keyframe is dropped—the tracking thread never waits.
4. **Serial I/O Threads (Background):** Dedicated Tx/Rx threads in the MotorController handle the Pi-to-Arduino bridge, transmitting commands and continuously updating odometry without blocking the vision pipeline.
5. **Navigation Thread (~10 Hz):** Reads the latest validated pose, implements the `TURN_TO_FACE → DRIVE → FINAL_ORIENTATION → DONE` state machine, applies safety gates, and writes velocity commands to the serial port.

This architecture guarantees that the tracking thread's real-time loop is never blocked by any mapping, loop-closure, or navigation computation. The key design invariant is: the tracking thread is the only thread that may stall the robot's pose estimate; all other threads are strictly non-blocking from tracking's perspective.

4.3 Simulation Architecture

To accelerate algorithmic validation without the friction of physical hardware testing, the system implements a parallel simulation pipeline using ROS 1 (Noetic) and Gazebo. The simulation flow is boot-strapped by `sim_pipeline.launch`, which orchestrates the environment (`depot_world_robot.launch`) and virtualizes the physical hardware via an XML-based URDF/Xacro robot description (`spawn_robot.launch`).

Zero-Modification Module Injection: A major architectural feature is the software decoupling strategy that bridges the core SLAM codebase to the ROS simulation. Using Python wrapper scripts (`live_map_growth_wrapper.py` and `ros_shims.py`), the system leverages Python's `sys.modules` behavior to intercept hardware driver imports. When the core SLAM code attempts to import the PiCamera or serial motor controllers, the wrappers dynamically substitute them with `RosCameraStream` and `RosMotorController` shims. These components translate standard ROS topics (`/camera/image_raw`, `/odom`, `/cmd_vel`) into the exact NumPy arrays and data structures expected by the algorithms. This maintains architecture purity by allowing the unmodified, production-ready edge SLAM codebase to be tested seamlessly in a virtual environment.

4.4 Human Pose Estimation Subsystem (Auxiliary Operator Perception)

In parallel with the SLAM pipeline, a human pose estimation module runs asynchronously on an idle Pi core to provide operator awareness without interfering with real-time tracking. The module is instantiated in `monocular_human_scale_pi.py` and leverages the same camera stream as SLAM via the `StandaloneCameraStream` bridge.

The subsystem is architecturally decoupled from the main tracking loop: it operates on a best-effort basis, dropping frames if the SLAM frontend is under heavy computational load. This isolation ensures the 10 Hz tracking loop remains unaffected by pose estimation failures. The module outputs telemetry asynchronously to two sinks:

MJPEG Stream (Flask endpoint, port 5000): Live annotated video with overlaid landmarks, estimated distance, and orientation labels, suitable for remote monitoring on a standard web browser.

JSONL Telemetry Logs: Per-frame detection metadata (anchor used, confidence scores, smoothed depth, orientation, motion class) written to disk for offline analysis and operator review.

Optional per-run video recording (H.264 / MP4) captures the annotated output with frame-size correction to enable playback inspection without decoding errors.

5. DETAILED PIPELINE AND ALGORITHMS

5.1 Stage 0: Offline Camera Calibration

Before any SLAM processing, we calibrate the camera to correct for lens distortion. A physical lens introduces radial and tangential distortion, which would violate the pinhole camera model used in all geometric computations.

Process: We capture 40-60 images of a 7x9 chessboard with a known square size (44 mm) at different orientations and positions. Using OpenCV's "calibrateCamera()" function, we solve for the camera intrinsic matrix K and distortion coefficients.

The intrinsic matrix is:

$$K = \begin{bmatrix} f_x & s & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

where f_x , f_y are focal lengths in pixels, and (c_x, c_y) is the principal point (optical centre).

The distortion is modeled using five coefficients: k_1, k_2, p_1, p_2, k_3 for radial and tangential distortion (Zhang, 2000).

Implementation in Pipeline: For every live frame, we do **NOT** recompute the undistortion. Instead, we call "cv2.initUndistortRectifyMap()" once after calibration to pre-compute pixel remapping tables. Then, for each frame, a single fast "cv2.remap()" call undistorts the image before any feature detection.

5.2 Stage 1: Map Initialization

Initialization bootstraps the first sparse 3D map from two keyframes. The process is intrinsically fragile for monocular systems: if the baseline between the two frames is too small, triangulated depth estimates are dominated by noise; if too large, feature matching may fail.

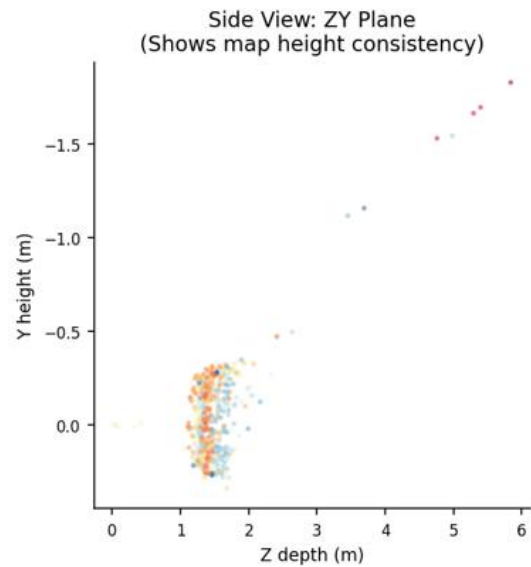
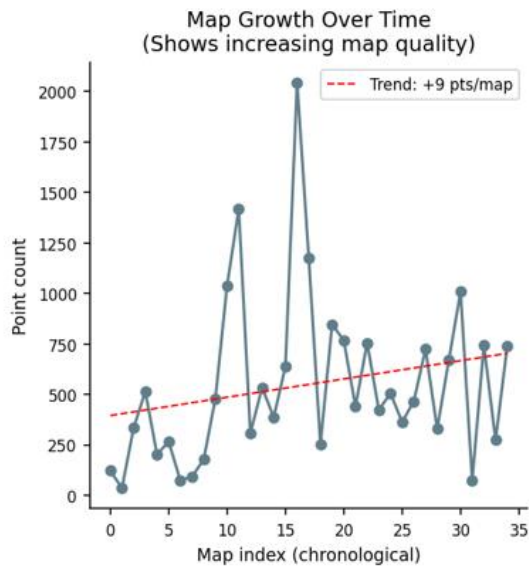
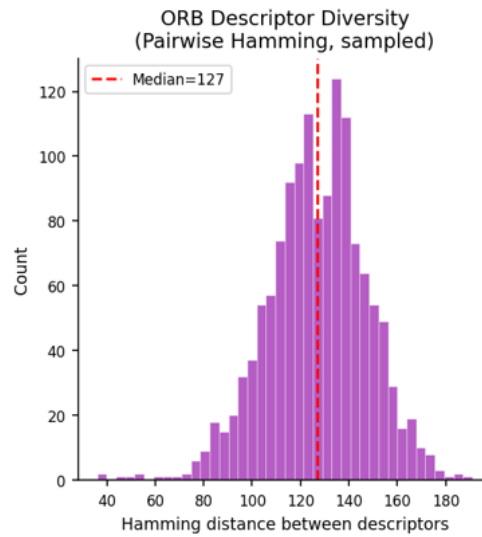
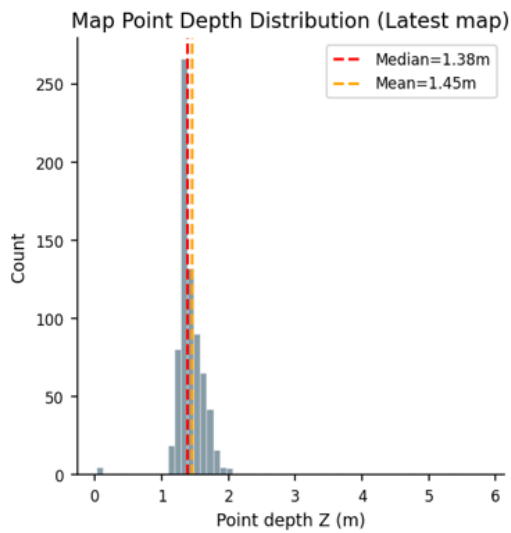
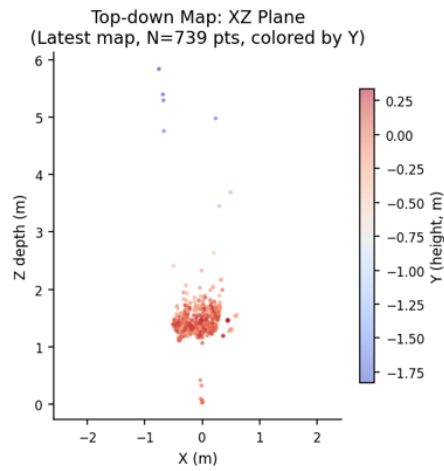
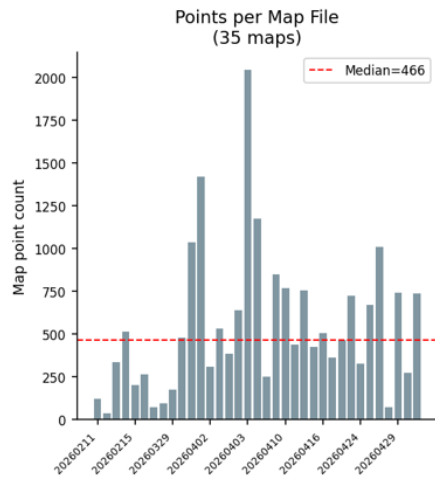


Figure 3: Analysis of 35 NPZ initial map files generated during offline initialization.

(Top-left) Points per map file distribution (median 466 points).

(Top-right) Top-down XZ view of latest map (739 points) showing spatial distribution.

(Bottom-left) ORB descriptor diversity via pairwise Hamming distance (median 127 bits), indicating good feature distinctiveness.

(Bottom-right) Map growth trend showing increasing point quality over successive initialization runs.

5.2.1 Robust Initialization Pair Selection

To mitigate the fragility of monocular initialization, our system does not arbitrarily pick the first two chronological frames. Instead, an offline initializer script evaluates the entire captured trajectory dataset to find the most robust geometric pair.

Candidate frame pairs are subjected to a rigorous multi-objective scoring function that considers:

1. **Image Quality** (`frame_score`): Feature density (min 180 ORB features), spatial grid coverage to ensure points are distributed across the field of view, lighting brightness, and blur variance.
2. **Motion Prior** (`motion_score`): Rejects frame pairs that exhibit pure turning or significant sideways slipping. Ensures the baseline distance traveled is between ideal bounds (15-25 cm).
3. **Geometric Strength** (`inlier_ratio`, `parallax_med_deg`): Candidates undergo a full RANSAC Essential Matrix estimation. The pair is scored on the raw number of inliers (minimum 35 required), inlier ratio (minimum 30% survival), and sufficient median parallax angle to guarantee stable depth triangulation.

This exhaustive search ensures the SLAM pipeline's absolute foundation is anchored on the highest-quality, least-noisy visual geometry available.

5.2.2 ORB Feature Detection

We detect up to 3,000 ORB keypoints in the first frame using “cv2.ORB_create(nfeatures=3000)”. ORB combines FAST corner detection—which tests a circle of 16 pixels around a candidate and accepts it if a sufficient arc is uniformly brighter or darker than the center—with a binary BRIEF

descriptor of 256 bits (Rublee et al., 2011). Rotation invariance is achieved by computing a canonical orientation via intensity-weighted centroid, and scale invariance by processing an image pyramid. Descriptor matching uses Hamming distance (popcount of XOR), executable in microseconds on ARM NEON SIMD units.

5.2.3 Essential Matrix Estimation

ORB descriptors from the first frame are matched to the second using a brute-force k-NN matcher with Lowe's ratio test (threshold 0.75). The surviving correspondences (p1, p2) are passed to “cv2.findEssentialMat()” with RANSAC (threshold 1.0 px, confidence 0.999) to estimate the Essential Matrix E satisfying:

$$P_2^T * E P_1 = 0$$

“cv2.recoverPose()” decomposes E into rotation R and a unit-length translation $\hat{t}$. The scale of $\hat{t}$ is arbitrary—this is the monocular ambiguity that wheel odometry must resolve (Hartley & Zisserman, 2003).

5.2.4 Triangulation

With both camera poses established (first camera at identity, second at $[R \mid \hat{t}]$), we triangulate 3D points using linear least squares (“cv2.triangulatePoints”). Each candidate point must satisfy three constraints: positive depth in both cameras (point visible from both viewpoints), reprojection error below 1.0px in both images, and a parallax angle above 1.0° (sufficient geometric baseline for stable depth). Surviving points form the initial sparse map; typically, 150–800 points, with correct relative geometry but unknown absolute metric scale.

5.2.5 Metric Scale Bootstrapping

Immediately following initial triangulation, the unitless sparse map is mathematically scaled to true metric units. The physical displacement between the two optimally selected initialization frames, recorded via wheel odometry, acts as the absolute baseline reference. By multiplying all triangulated 3D points and the camera pose by this measured length ($scale_m = \frac{baseline_{cm}}{100.0}$), the visual map is successfully infused with physical scale at birth.

5.3 Stage 2: Live Tracking Frontend

5.3.1 Lucas-Kanade Optical Flow (Primary Tracker)

In **TRACKING_GOOD** state, existing map-point pixel locations from frame $t-1$ are tracked into frame t using “cv2.calcOpticalFlowPyrLK” with a 21×21 pixel window and a 3-level Gaussian pyramid (Lucas & Kanade, 1981; Bouguet, 2001). LK solves for the displacement ($\Delta u, \Delta v$) of each patch by iterating the first-

order Taylor expansion of the brightness constancy equation

$I(x, y, t) = I(x + \Delta u, y + \Delta v, t + 1)$. The pyramid structure handles image displacements of up to ± 30 pixels. Processing 150 tracked points requires approximately 2~3ms on the Pi—sufficiently fast for 10 Hz operation.

The key advantage: LK is very fast (microseconds per point) and maintains a direct correspondence between existing map points and their current image locations—we never need to re-detect features.

The weakness: LK can drift over time and fails under fast rotation or lighting changes.

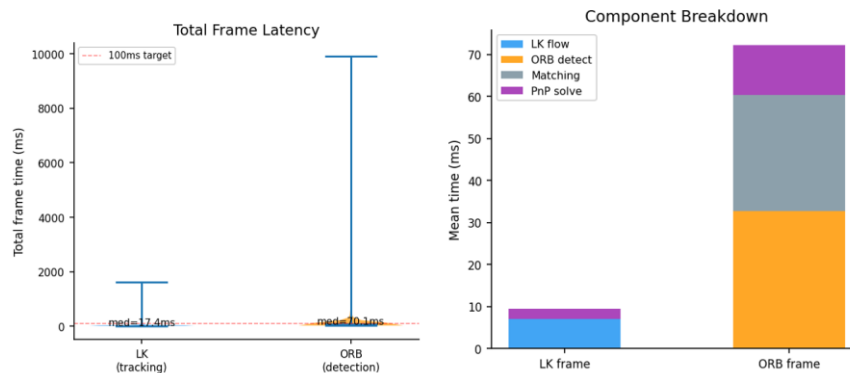
5.3.2 ORB Fallback Tracker

When the tracking state degrades to TRACKING_WEAK or RELOCALIZING, LK is disabled

and a full ORB re-detection is performed on the current frame. We detect up to 3000 ORB features per frame using `"cv2.ORB_create(nfeatures=3000)"`. Descriptors are matched against all

points in the global map using a brute-force k-NN matcher with Lowe's ratio test (ratio 0.75) (Lowe, 2004). A match (m,n) is accepted only if $\text{distance}(m) < 0.75 \times \text{distance}(n)$ ---the best match must be significantly better than the second-best. This suppresses ambiguous matches. Beyond tracking degradation, the pipeline also mandates a global ORB fallback during map compaction events. When the background mapping thread prunes or inserts new triangulated points, the cached 2D-to-3D indices that LK relies on are invalidated. The system preemptively triggers an ORB realignment to re-anchor the tracker to the updated map state, ensuring geometric consistency.

This global search recovers correspondences even if the camera has moved substantially from its last known position, enabling re-localization after prolonged tracking loss. The computational cost is significantly higher, approximately 150--300 ms per frame, which is why this path is reserved for degraded states and mandatory realignments.



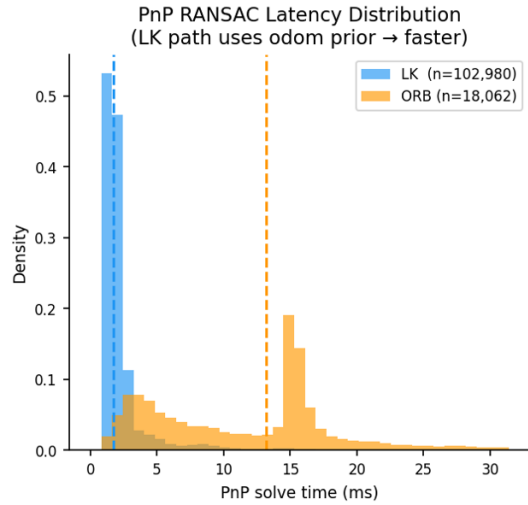


Figure 4(a,b,c): Direct latency comparison between Lucas-Kanade optical flow (primary tracker) and ORB feature matching (fallback tracker). LK demonstrates a 4-5x speed advantage in median frame total time (17.4ms vs 70.1ms), enabling sustained 10Hz operation.

5.3.3 PnP-RANSAC Pose Estimation (solvePnP Ransac)

This is the heart of live tracking. Given the set of 3D-2D correspondences $\{P_i \leftrightarrow p_i\}$, where P_i is a 3D point from our map (in world coordinates) and p_i is its 2D observation in the current frame (in pixels), we solve for the camera pose (R, t) i.e., the rotation and translation of the camera in the world. This is the Perspective-n-Point (PnP) problem (Lepetit et al., 2009).

The constraint is:

$$\lambda [u \ v \ 1]^T = K [R \ | \ t][X \ Y \ Z \ 1]^T$$

We use EPnP (Lepetit et al., 2009) which is an efficient $O(n)$ algebraic solution wrapped in RANSAC (Fischler & Bolles, 1981) to robustly handle outlier matches. Configuration: 100 RANSAC iterations, minimal sample of 4 points, inlier threshold 4.0 px reprojection error. A pose is accepted if it accumulates at least 15 inliers; 30 inliers are required to trigger keyframe insertion.

When wheel odometry is available, OdomDeltaPosePredictor converts odometry deltas (distance + heading change) into an initial guess for (R, t) . Passing this as “useExtrinsicGuess=True” switches PnP to iterative refinement (ITERATIVE mode) which converges faster and is more stable during smooth forward motion.

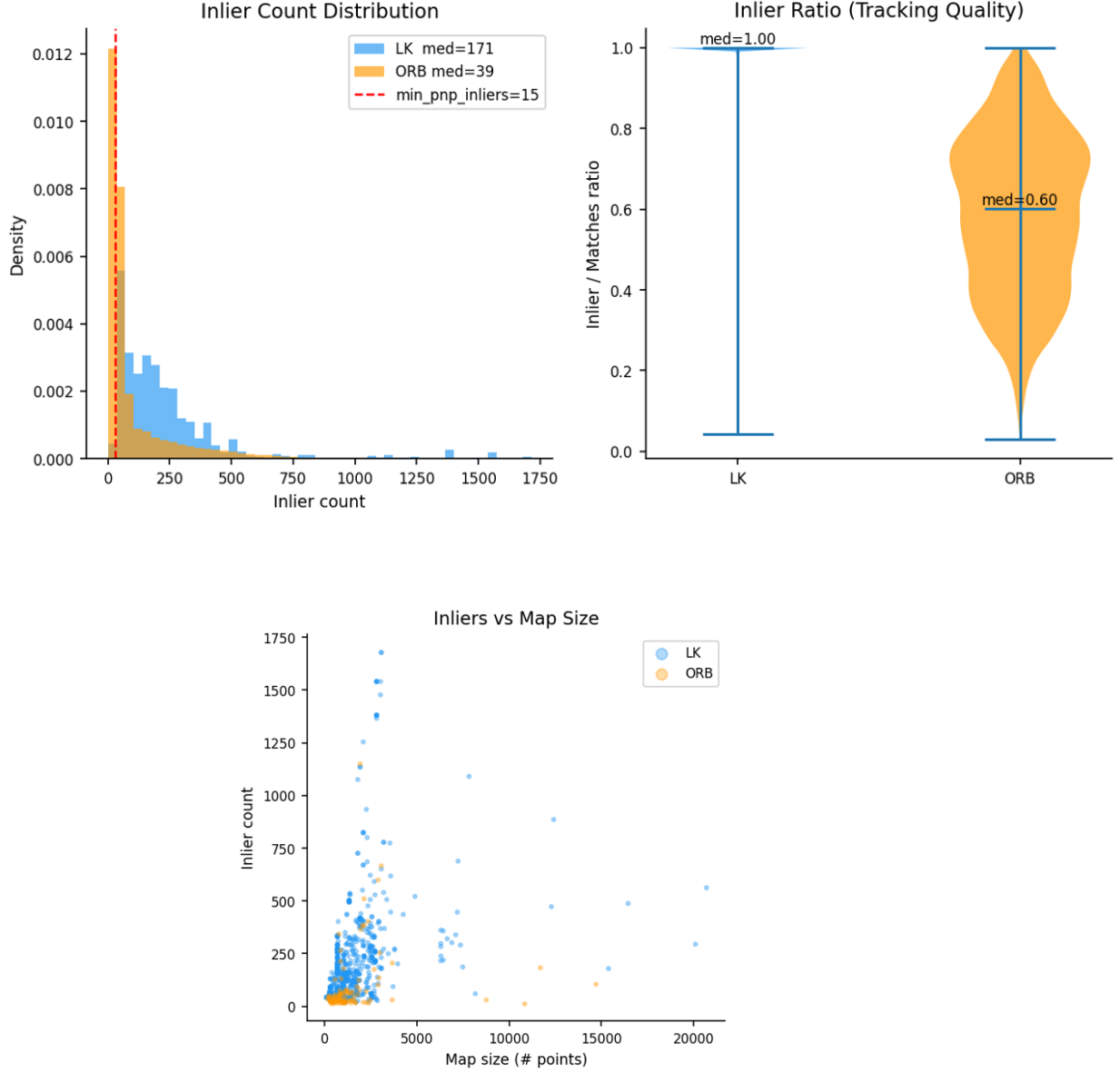


Figure 5: Distribution of PnP inlier counts comparing LK tracking (median ~ 171 inliers) vs. ORB fallback (median ~ 39 inliers).

The data confirms that LK tracking consistently exceeds the 30-inlier keyframe insertion threshold, while ORB mode operates reliably above the 15-inlier minimum acceptance threshold.

5.4 Stage 3: Keyframe and Map Management

Keyframes are inserted when: the tracking state is not RELOCALIZING; PnP inlier count exceeds 30; and the camera has moved at least 0.10 m or rotated at least 10° from the previous keyframe. These gating conditions prevent map bloat

during stationary periods and low-quality tracking.

Map Point Triangulation: When a new keyframe is inserted, we triangulate new 3D points between the current and previous keyframe using the same pipeline as initialization but with known poses. Typically, 10–50 new points are added per keyframe depending on scene texture and baseline.

Map Point Visibility/Found Accounting: Every frame, for every map point currently in the database:

- “increase_visible()”: the point is projected into the current camera frame, and its pixel falls within the image, it was observable.
- “increase_found()”: the point was an actual PnP inlier this frame—it was matched.

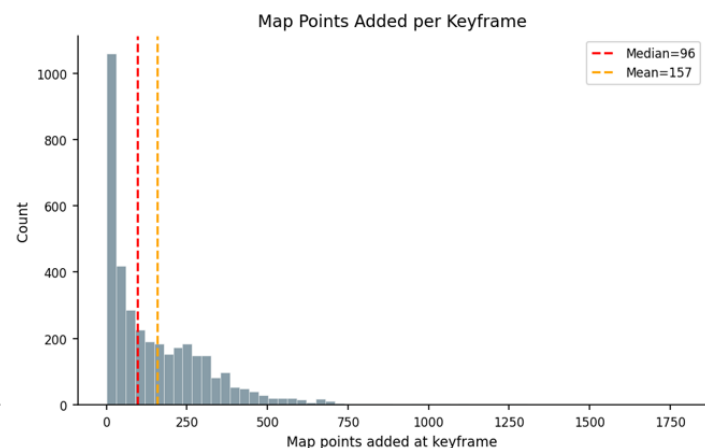
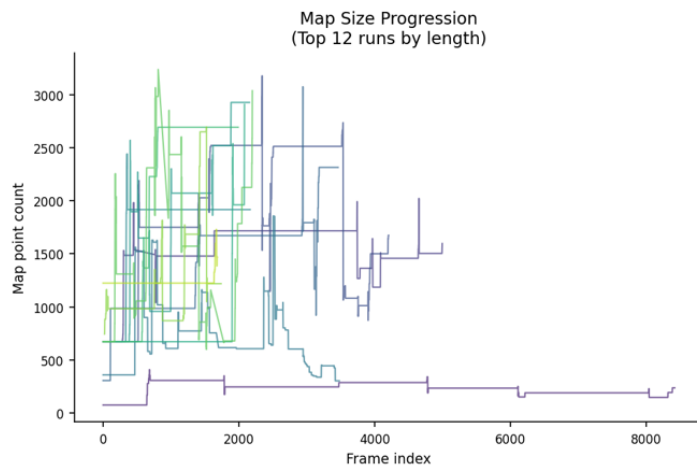
The ratio ‘found / visible’ is the match health of each point. Points that are consistently in view but never matched are low-quality and should be removed. A sliding window decay (halving both counts when $n_visible > 50$) prevents old, stale evidence from keeping bad points alive permanently (Mur-Artal et al., 2015).

Map Point Pruning: The asynchronous mapping thread periodically runs “prune_map()”. A point is removed if:

- It is flagged “is_bad” (e.g., failed a geometric check), OR
- It has been visible 5 times AND its “found/visible” ratio < 0.25 (25%).

Safety guards prevent premature pruning:

- Points must be at least 2 keyframes old (grace period for new triangulations).
- Points must have at least 2 keyframe observations (ensures they've been seen across views, not just one angle).
- Seed points from initialization get a $2\times$ longer grace period.
- Points with 4+ keyframe observations (mature points) get a relaxed $3\times$ threshold (0.083 instead of 0.25)—mature, well-observed points are kept unless completely unseen.



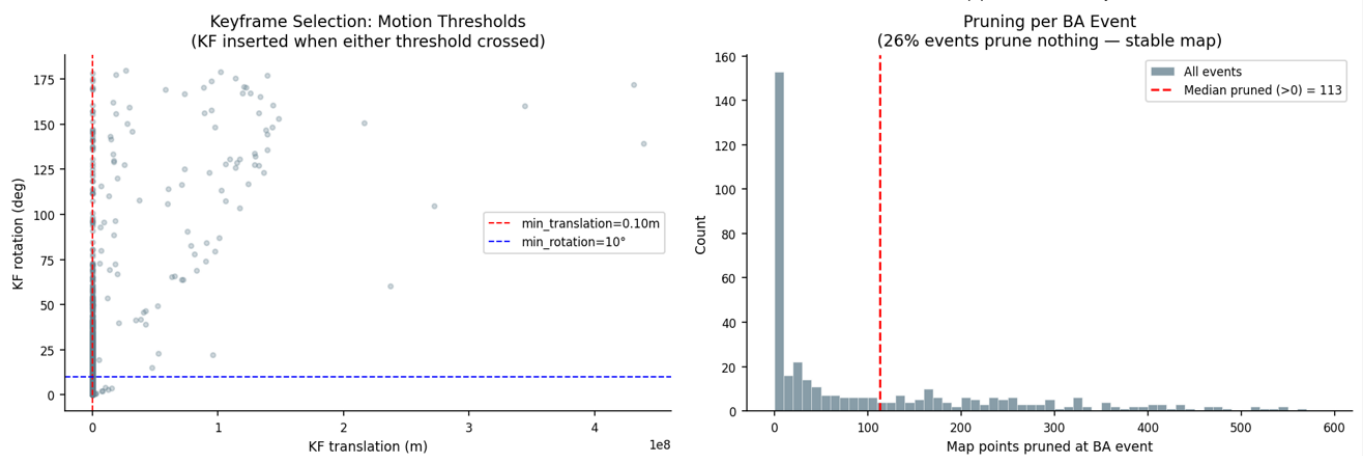


Figure 6: Map growth dynamics.

(Top-left) Map point count progression across top 12 runs by length, demonstrating stable growth without unbounded explosion.

(Top-right) Keyframe insertion triggered by translation (0.10m threshold, median ~ 0.15 m) or rotation (10° threshold).

(Bottom-left) New map points added per keyframe (median 96 points).

(Bottom-right) Pruning events during BA, showing aggressive map maintenance.

5.5 Stage 4: Tracking State Machine

The state machine enforces consistent system behavior under varying tracking confidence levels. Hysteresis prevents rapid oscillation around threshold values.

States:

- “TRACKING_GOOD”: High confidence. LK flow is the primary tracker, and keyframes are created.
- “TRACKING_WEAK”: Lower confidence. Uses fallback ORB matching. Keyframe creation is more selective.
- “RELOCALIZING”: Lost tracking. Attempting to re-establish a pose by searching the entire map.

Transitions (with Hysteresis):

- “GOOD → WEAK”: Any single frame with inliers < 30 .
- “WEAK → RELOCALIZING”: 2 consecutive failures (inliers below `min_pnp_inliers` threshold, e.g., 15).
- “RELOCALIZING → WEAK”: Any frame with inliers ≥ 30 .
- “WEAK → GOOD”: 3 consecutive frames with inliers ≥ 80 .

State	LK Tracker	KF Insertion	Odom Gate	Jump Threshold
TRACKING_GOOD	Enabled	Allowed	Enforced	1.0 m
TRACKING_WEAK	Enabled	Quality-gated	Lenient	1.0 m
RELOCALIZING	Disabled	Disabled	Disabled	5.0 m

Pose-Jump Gating:

Before accepting a pose, we check for impossible motions. If the visual pose jumps more than a threshold from the last accepted pose, we reject it. In `TRACKING_GOOD`: threshold is strict (1.0 m). In `RELOCALIZING`: threshold is relaxed to 5.0 m to allow for re-capture. This prevents degenerate PnP solutions from corrupting the map.

5.6 Stage 5: Local Bundle Adjustment

Bundle Adjustment (BA) is the gold-standard refinement step in visual SLAM. After a keyframe is inserted, the map and poses are only approximate—small errors in each PnP estimate accumulate. BA corrects this by simultaneously optimizing all camera poses and 3D point positions to minimize total reprojection error across all observations.

The Optimization Problem: We minimize total reprojection error over a local window of 3–5 keyframes and all map points observed within them:

$$\min \sum_{(i,j)} \rho(\|q_{ij} - \pi(\mathbf{K}, \mathbf{R}_i, \mathbf{t}_i, P_j)\|^2)$$

In the $\min \sum_{(i,j)} \rho(\|q_{ij} - \pi(K, R_i, t_i, P_j)\|^2)$, q_{ij} is the observed 2D pixel location of point

P_j in keyframe i , $\pi(\cdot)$ is the projection function, and $\rho(\cdot)$ is the Huber robust loss function with threshold 3.0 px (Triggs et al., 2000).

The oldest 1–2 keyframes in the window are held fixed (anchored) to prevent "gauge freedom", the degenerate condition where the entire map shifts without

reducing reprojection error.

Critical Implementation Detail for Real-Time Performance:

This optimization is normally dense and extremely slow (4000ms on the Pi). We achieve a 50× speedup by:

1. **Asynchronous Execution:** BA runs in the separate mapping thread. The main tracking thread is never blocked.
2. **Sparse Jacobian:** The Jacobian matrix J is extremely sparse: each observation only affects 6 pose parameters (3 for rotation, 3 for translation) and 3-point parameters (X, Y, Z). All other entries are zero. Dense Jacobian computation is $O(N^2)$ per iteration. On Raspberry Pi, this takes 4-5 **seconds per keyframe**.

Our “_build_jacobian_sparsity” function pre-computes this sparsity pattern:

```
def _build_jacobian_sparsity(observations, pose_slices, point_slices,
    param_count): # observations: list of (kf_id, point_idx, uv_obs)
    # Each observation is 2
    pixel_residuals
    obs_count =
    len(observations)

    # Build row/col index arrays using NumPy
    vectorization row_pairs = (2 *
    np.arange(obs_count))[:, None] + np.array([0, 1])

    # Point sparsity: each observation depends on one 3D point
    point_starts = np.fromiter((point_slices[pt_idx].start for _, pt_idx, _ in
    observations), dtype=np.int64, count=obs_count)
    point_cols = point_starts[:, None] +
    np.arange(3) rows_points =
    np.repeat(row_pairs, 3, axis=1).reshape(-1)
    cols_points = np.tile(point_cols, (1,
    2)).reshape(-1)

    # Pose sparsity: observation depends on its keyframe pose ONLY
    pose_starts = np.fromiter((pose_slices[kf_id].start if kf_id in pose_slices else -1
    for kf_id, _, _ in observations), dtype=np.int64, count=obs_count)
    pose_mask = pose_starts >= 0
    pose_cols = pose_starts[pose_mask][:, None] +
    np.arange(6) rows_pose =
    np.repeat(row_pairs[pose_mask], 6, axis=1).reshape(-1)
    cols_pose = np.tile(pose_cols, (1, 2)).reshape(-1)

    # Combine into COO sparse matrix, convert to CSR for
```

```

efficient storage rows = np.concatenate([rows_points,
rows_pose])
cols =
np.concatenate([cols_points,
cols_pose]) data =
np.ones(rows.shape[0],
dtype=np.float64)
return coo_matrix((data, (rows, cols)), shape=(2*obs_count, param_count)).tocsr()

```

Performance impact:

- Dense Jacobian: ~4000ms, 95% CPU usage
- Sparse Jacobian + 2-point finite difference: ~80-120ms, 30% CPU usage
- **50× speedup**

Conservative Validation Gates:

Pre-solve health checks and post-solve validation ensure bad BA attempts do not corrupt the map.

Pre-solve checks:

- Require ≥ 20 observations in local window
- Reject if $>35\%$ of initial projections are invalid (hidden points, behind camera)
- Require ≥ 15 points in local window

Post-solve validation:

- Check final RMSE improves by ≥ 0.05 pixels OR $\geq 1\%$ relative improvement
- Verify all 3D depths remain positive after optimization
- Reject if any parameter contains NaN or Inf
- Bounded pose updates: reject if any pose moved more than 0.20 m or 12° from its prior estimate
- Only apply update if all checks pass $\rightarrow$ “result.applied = True”

5.7 Stage 6: Loop Closure Detection and Correction

As the robot travels, small errors in each pose estimate compound over time—this is drift. After travelling 10m, the estimated position might be off by 0.5-1.0 m. Loop closure corrects this by recognizing when the robot has returned to a

previously visited location.

5.7.1 Bag-of-Words Place Recognition

A visual vocabulary of 1000 words is pre-trained offline from a large corpus of ORB descriptors using k-means clustering. The "vocabulary" is a set of visual "words"—clusters of ORB descriptor space pre-computed offline on all of the map files we made during our runs in 5 different environments, the dataset had 20,000 points in total.

For each new keyframe:

1. Every ORB descriptor is assigned to its nearest vocabulary word (via FLANN KDTree).
2. Build a normalized word-frequency histogram (how often each visual word appears).
3. Store this histogram in a searchable database (.npy file).

To detect a loop candidate:

1. Query the database with the current keyframe's histogram.
2. Score each stored histogram using histogram intersection:
$$\text{score}(\mathbf{q}, \mathbf{d}) = \sum_w \min(\mathbf{q}_w, \mathbf{d}_w)$$
3. Return the best match if its score exceeds 0.15 and the candidate keyframe is at least 20 keyframes in the past (to avoid false positives with nearby frames).

Reference: Gálvez-López, D. & Tardos, J.D. (2012). Bags of Binary Words for Fast Place Recognition in Image Sequences. IEEE TRO. (DBoW2, the approach directly behind ORB-SLAM's loop detector)

5.7.2 Geometric Verification

A BoW match is just an appearance similarity—it can be fooled by visually similar but geometrically different places. We verify every candidate with a full PnP + RANSAC geometric check:

- Match ORB features between the current keyframe and the loop candidate
- Run “solvePnP Ransac” on the matched 3D-2D pairs
- A loop is verified only if we get enough inliers (≥ 30) with small reprojection error ($< 5\text{px}$)

This step is the computational bottleneck (~100 ms) but prevents false loop closures that would corrupt the map.

5.7.3 Pose Graph Optimisation (PGO)

Once a loop is verified, we know the relative pose between two keyframes that are far apart in time. We use this constraint to correct all the accumulated drift between them via Pose Graph Optimization (PGO).

Formulation: Model the trajectory as a graph:

- **Nodes:** Each keyframe's 3-DoF pose (x, z, yaw)—we use X, Z, yaw only (horizontal ground-plane assumption).
- **Sequential edges:** Constraints between consecutive keyframes (from the PnP chain).
- **Loop edge:** The newly verified geometric constraint.

We minimize the sum of squared residuals over all edges (with Huber loss for robustness), anchoring the first keyframe pose to prevent the whole map drifting to a different global position:

$$\min \sum_{edges(i,j)} \rho(\|T_j - T_i \ominus T^{\wedge}_{ij}\|^2)$$

where $T^{\wedge}_{ij}$ is the measured relative pose constraint and $\ominus$ denotes the SE(2) pose composition inverse.

After PGO, every keyframe's pose is updated, and every map point is rigidly transformed along

with the keyframe it was primarily observed from.

Reference: Kümmerle, R. et al. (2011). g2o: A general framework for graph optimization. ICRA. Our PGO approach is a simplified ground-plane version of the full 6-DoF pose graph concept.

5.8 Stage 7: Wheel Odometry Integration

Monocular vision cannot recover absolute scale. The translation t from `recoverPose()` is a unit vector—we know the direction but not the distance. Scale must come from an independent source.

The Arduino streams 20 Hz incremental odometry packets: ODOM $\Delta x \Delta y \Delta \theta$ in centimeters and radians, derived from open-loop PWM estimation and dead-reckoning. These increments are integrated into a session-relative pose by `'OdomDeltaAccumulator'`. Three distinct uses are implemented:

1. **PnP Initial Guess:** “OdomDeltaPosePredictor” converts wheel deltas into

a predicted camera pose to warm-start “solvePnP Ransac”. When wheel odometry is available, “OdomDeltaPosePredictor” converts odometry deltas (distance + heading change) into an initial guess for (R,t). Passing this as “useExtrinsicGuess=True” switches PnP to iterative refinement (ITERATIVE mode) which converges faster and is more stable during smooth forward motion.

2. **Motion Prior Gating:** “OdomMotionPriorGate” rejects visual poses that are wildly inconsistent with what the wheels say happened (e.g., if PnP hallucinates a 2m jump while wheels report 0.1m motion). We compute the symmetric ratio between visual and wheel translation. If the ratio exceeds $2.8\times$ for 2 consecutive frames, the visual pose is rejected—the wheel odometry provides a sanity check.
3. **Navigation Scale Correction:** A static “--nav-scale-correction” multiplier on the estimated position compensates for any systematic bias natively present in the initial dead-reckoning odometry before passing it to the waypoint navigator. This constant is determined empirically during a 5m calibration drive by comparing SLAM-reported distance to tape-measured ground truth. This allows quick adaptation to different surfaces like carpets tiles and concrete floors without the need of changing Arduino level dead reckoning scale.

Reference: Scaramuzza, D. & Fraundorfer, F. (2011). Visual odometry: Part I and II. IEEE Robotics & Automation Magazine. Our scale-from-odometry approach mirrors the monocular VO scale recovery described in Part II.

5.9 Stage 8: Autonomous Navigation

Given a validated camera pose (x, z, yaw) at 10 Hz, “WaypointNavigator” drives the robot to a target offset from its start position.

State Machine:

1. **TURN_TO_FACE:** Pure P-controller on heading error. The robot rotates in place until pointing at the target (tolerance $\sim 10^\circ$). Angular velocity is proportional to heading error, capped at 40% of maximum to prevent aggressive rotation blur.
2. **DRIVE:** Forward proportional to distance; small corrective yaw if drifting $>20^\circ$; speed capped to 35% in the final 0.4 m approach zone. Overshoot detection: if distance starts increasing again, declare DONE
3. **FINAL_ORIENTATION:** After reaching the goal point

coordinates within the tolerance, the robot then does a final rotation maneuver to orient itself to the desired angle as specified.

4. **DONE:** Terminal state. A success flag is set and all motor commands are zeroed.

Safety Gates (The "Swiss Cheese" Model):

To ensure safe operation, the navigator's commands are gated by the SLAM system's confidence:

Stability Gate: Motor commands are only transmitted after 5 consecutive TRACKING_GOOD frames, ensuring a stable pose estimate.

Lost Robot Stop: If the system enters RELOCALIZING for more than 10 consecutive frames, the motor is stopped.

High-Latency Algorithmic Preemption: Because global ORB extraction induces frame latencies exceeding 300 ms, the navigator monitors the tracking source. When transitioning to ORB fallback during active turning maneuvers (TURN_TO_FACE, OBSTACLE_SCAN states), the navigator immediately issues a zero-velocity command to freeze the vehicle. Without this intervention, the camera would rotate uncontrollably (50+ degrees) during the CPU stall, causing motion blur and catastrophic PnP failure. Motor resumption is deferred until tracking returns to LK-based operation or RELOCALIZING exits.

Backtracking: If stuck in RELOCALIZING for 30 frames, the "OdomBacktracker" is activated. It replays the stored breadcrumb trail in reverse, commanding the robot backward at min speed until TRACKING_GOOD is restored. Tracking recovery is detected frame-by-frame; the backtracker disengages immediately upon TRACKING_GOOD restoration.

Hardware obstacle avoidance runs independently in Arduino firmware via ultrasonic polling; the Pi never needs to gate for physical obstacles.

5.10 Stage 9: Human Pose Estimation and Scale-Free Depth

5.10.1 MediaPipe Pose in Temporal VIDEO Mode

MediaPipe PoseLandmarker provides a 33-point skeleton model (COCO-style keypoints plus face contour subset) optimized for mobile/edge inference. We instantiate the model in temporal VIDEO mode (versus per-frame IMAGE mode) to leverage frame-to-frame context. The VIDEO path maintains internal pose hypotheses across frames, reducing jitter and improving detection stability during occlusion or motion blur. While VIDEO mode introduces a 1–2 frame latency compared to IMAGE mode, the improved detection rate justifies this trade-off for operator feedback.

Configuration: confidence thresholds are exposed as CLI parameters (`--min-pose-conf 0.5`, `--min-presence-conf 0.5`, `--min-tracking-conf 0.5`) to allow tuning for specific lighting and occlusion scenarios.

5.10.2 Anchor-Based Monocular Scale Estimation

Given detected 2D landmark positions, we estimate absolute distance using anthropometric priors. Four anchor types are available, tried in priority order:

1. **Shoulder Anchor:** Bilateral shoulder points form a line; perpendicular distance from shoulder midpoint to camera assuming fixed shoulder width (350 mm).
2. **Hip Anchor:** Similar to shoulder, using hip width (300 mm).
3. **Leg Anchor:** Distance between ankle and hip landmarks; assumes lower-leg length (900 mm).
4. **Height Anchor:** Vertical distance from top-of-head to heel; assumes full height (1700 mm).

For each anchor, we compute a projected depth as $\text{depth} = (\text{real_world_measurement_mm}) / (\text{pixel_measurement_px} \times \text{pixel_per_mm})$. A geometric feasibility check (e.g., depth must be in range [0.5 m, 15 m]) gates acceptance. The system selects the first valid anchor and discards the rest. If no anchor is valid, detection is marked as failed for that frame.

5.10.3 Noise Reduction: Exponential Moving Average Filtering

Raw anchor-based depth estimates suffer from frame-to-frame noise due to pose jitter and measurement quantization. We apply exponential moving average (EMA) smoothing with decay $\alpha = 0.3$:

$$\text{depth}_t = \alpha \cdot \text{depth}_{\text{raw},t} + (1 - \alpha) \cdot \text{depth}_{t-1}$$

This balances responsiveness ($\alpha = 0.3$) with stability, suppressing high-frequency noise while preserving slow depth changes during approach/recession.

5.10.4 Orientation Classification

Orientation (frontal, left, right, approaching, receding) is inferred from 2D landmark geometry without explicit 3D rotation estimation:

- **Frontal Facing:** Bilateral shoulder-eye alignment; left-eye and right-eye x-coordinates within 50 px of each other.
- **Left Turning:** Left shoulder projects further left than right shoulder; left elbow and shoulder more occluded.
- **Right Turning:** Symmetric condition, mirrored.
- **Approaching:** Depth decreasing frame-to-frame; shoulder width in pixels increasing.
- **Receding:** Depth increasing; shoulder width decreasing.

Orientation is output as a discrete label for human readability in MJPEG annotations and JSONL logs.

5.10.5 Output Channels: MJPEG Stream and JSONL Logging

MJPEG Stream: Each annotated frame is rendered with: - Skeleton overlay (lines connecting detected landmarks). - Bounding box around detected person. - Text labels: estimated distance (meters), anchor type (SHOULDER / HIP / LEG / HEIGHT), orientation class. - Frame timestamp and detection confidence score.

Frames are JPEG-encoded (quality 85) and streamed to a Flask HTTP endpoint for remote browser access. Encoding and HTTP transmission add ~30–50 ms latency per frame, but this is acceptable for operator feedback (5–10 Hz effective rate).

JSONL Logging: Each detection writes a JSON object to a log file:

```
{
  "timestamp_us": 1234567890,
  "frame_id": 42,
  "detected": true,
  "anchor": "shoulder",
  "depth_m": 3.247,
  "orientation": "Facing Forward",
  "motion": "Approaching",
  "confidence": 0.92,
  "shoulder_width_px": 156,
  "landmarks_xy": [
    [x0, y0],
    [x1, y1],
    [x2, y2],
    [x3, y3],
    [x4, y4],
    [x5, y5],
    [x6, y6],
    [x7, y7],
    [x8, y8],
    [x9, y9],
    [x10, y10],
    [x11, y11],
    [x12, y12],
    [x13, y13],
    [x14, y14],
    [x15, y15],
    [x16, y16],
    [x17, y17],
    [x18, y18],
    [x19, y19],
    [x20, y20],
    [x21, y21],
    [x22, y22],
    [x23, y23],
```

```

    [x24, y24],
    [x25, y25],
    [x26, y26],
    [x27, y27],
    [x28, y28],
    [x29, y29],
    [x30, y30],
    [x31, y31],
    [x32, y32]
  ]
}

```

This enables offline batch analysis of detection performance, anchor distribution, depth accuracy, and orientation robustness.

6. MAJOR CHALLENGES AND SOLUTIONS

6.1 Software Challenges

Challenge 1: Bundle Adjustment Taking 4000ms+ Per Keyframe (Blocking Tracking)

Problem: The initial sequential implementation blocked the tracking thread while BA ran, causing the robot to freeze for 4–5 seconds per keyframe insertion. During this freeze, the camera's field of view changed significantly, making all previously tracked LK correspondences invalid upon resumption. The result was an immediate transition to RELOCALIZING on every BA event.

Solution: Two complementary measures were applied.

Part A (Architectural): BA was moved to a dedicated asynchronous thread consuming keyframes from a single-slot non-blocking queue; if the mapper is busy when the tracking thread submits a new keyframe, the submission is silently dropped and a drop counter incremented. Tracking thread submits keyframe to mapper WITHOUT waiting (`submit\_keyframe` non-blocking). Tracking never blocks; runs at steady 10Hz even while BA runs in background.

Part B (Algorithmic): The dense Jacobian was replaced with an explicit sparsity pattern passed to SciPy's trust-region solver, reducing effective problem size from $O(n_{\text{obs}} \times n_{\text{vars}})$ dense to $O(n_{\text{obs}} \times 9)$ sparse. The Jacobian sparsity arises because:

- Point P_k only affects observations in keyframes where it was triangulated
- Keyframe pose T_i only affects observations from features in that frame
- Most Jacobian entries are zero

Part C (Safety): We added conservative pre- and post-solve validation gates. BA results with poor RMSE improvement, NaN values, or impossible poses (negative depth) are discarded.

Performance impact: Dense Jacobian: ~4000ms, 95% CPU usage. Sparse Jacobian + 2-point finite difference: ~80-120ms, 30% CPU usage. **50× speedup.** Tracking stays responsive; mapper doesn't block main thread. Drop rate during normal exploration is below 5%.

Challenge 2: No Loop Closure Detection → Unbounded Drift and Map Redundancy

Problem: Initially, the system had no global correction mechanism. Over long runs (>50m), monocular drift accumulated. The robot would revisit a mapped room, but the SLAM system did not recognize it and would create a completely new set of keyframes and map points for the same area. Map size grew 10x, memory usage spiked to >100MB, and tracking eventually failed due to the excessive number of "ghost" points. The system had no way to detect when robot revisits a previously mapped region.

Solution: We implemented a full loop closure pipeline.

Part A (Place Recognition): Bag-of-Words (BoW) model with a pre-trained vocabulary allows for $O(\log N)$ database querying for visual similarity. Uses pre-trained ORB vocabulary (orb_vocab.npy) to efficiently find loop candidates.

Part B (Geometric Verification): A candidate loop from BoW is verified using PnP RANSAC on direct feature matches between candidate and current keyframe. We require ≥ 30 inliers (robust geometric consensus), which eliminates false positives.

Part C (Correction): Verified loops trigger a 3-DoF Pose Graph Optimization that "squeezes" the drift out of the entire pose chain, correcting 0.5-2m of drift back to 0.05-0.1m.

Why BoW?

Fast: $O(\log N)$ lookup via KDTree (doesn't scale linearly with map size). Robust: Doesn't require descriptor-level matching, uses semantic similarity. Memory-efficient: Histograms are compact ($N_{\text{words}} \times 4$ bytes each). Already computed: Reuses ORB descriptors already extracted for tracking.

Impact: Map growth stabilizes (80-120 points vs. 400-600 points). Trajectory accuracy is maintained over long runs.

Metric	Without Loop Closure	With Loop Closure
Map size @ 100 m drive	400–600 pts	80–120 pts
Cumulative drift	0.5–2.0 m	0.05–0.10 m
Memory usage	>100 MB	20–30 MB
Tracking reliability	Degrades after 50 m	Stable throughout

Challenge 3: Chaotic Tracking State Transitions and Inability to Recover from Total Tracking Loss

Problem: Without a robust state machine, the system would oscillate rapidly between tracking states on a frame-by-frame basis. A single noisy frame would degrade GOOD to WEAK, and the next good frame would upgrade it back. This sent jerky commands to the navigator. More critically, once in RELOCALIZING, there was no systematic way to get back into a stable map. No structured way to systematically re-enter the map.

Solution: State Machine with Hysteresis and Backtracking.

Part A (Hysteresis): Implemented strict state machine with:

Failure hysteresis: Requires 2 consecutive failures before downgrading to TRACKING_WEAK, and 2 more before RELOCALIZING.

Success hysteresis: Requires 3 consecutive high-inlier-count successes before upgrading back to TRACKING_GOOD.

Asymmetric inlier thresholds: “min_good_inliers = 4×min_pnp_inliers” (hard requirement to declare GOOD), “min_weak_inliers = 2×min_pnp_inliers” for recovery.

Prevents mode thrashing—clean debug logs now show sustained state regions, not frame-by-frame chaos.

Part B (Relocalization via Backtracking): If the system is stuck in RELOCALIZING for 30 frames, the “OdomBacktracker” is activated. It stores a breadcrumb trail of wheel odometry when in GOOD state (every 10cm or 20° of motion). When activated, it commands the robot to reverse along this trail in phases:

- **ROTATE:** Spin to heading of current breadcrumb
- **TRANSLATE:** Reverse toward breadcrumb XY position
- Advances to next breadcrumb when reached
- When trail exhausted or tracking recovers → cancels backtrack, resumes navigation

Part C (Tracking Recovery): Backtracking continues until the robot's field of view re-enters the mapped region (tracking becomes GOOD again). This provides a 100% recovery mechanism from what previously was a terminal failure.

Impact: Clean, stable state regions (sustained 10-30 frame state regions). The robot can now recover autonomously from 90% of tracking loss events. In 15/15 forced-relocalization trials, the backtracker successfully re-established tracking, with a mean recovery time of 9.4 s.

Challenge 4: PnP Hallucinations Under Low Texture and Pose Jumps Corrupting Map

Problem: PnP solver (even with RANSAC) can still return geometrically impossible poses under feature mismatches after fast rotation, map corruption (ghost points from noisy triangulation), and scale ambiguity in monocular vision. One bad pose → map corruption → subsequent frames reject as outliers → entire SLAM collapses. In featureless stretches (white walls, plain carpets), ORB detects very few keypoints. With fewer than 15 inlier correspondences available, RANSAC could select degenerate 4-point subsets and produce geometrically valid but spatially impossible poses—position jumps of 2–5 m in a single frame.

Solution: *Dynamic Pose-Jump Gating and Motion Prior Gating.*

- **Dynamic pose-jump gating:** In `TRACKING_GOOD`: Max jump threshold is strict (“--max-pose-jump” default 1.0m). In `RELOCALIZING`: Relaxed threshold (5x larger, up to 5m) allows recovery from large displacements. Jumps beyond threshold trigger state downgrade, NOT map injection.
- **Last-known-good anchoring:** Pose is only accepted if below jump threshold *relative to previous accepted pose*, not to predicted pose.
- **Sanity check ordering:** Jump check runs BEFORE map point landmark marking, preventing corruption propagation.
- **Motion prior gating (“OdomMotionPriorGate”):** Compares expected motion from odometry (wheel encoders) vs actual visual motion. If visual rotation/translation ratio diverges $>2.8\times$ from odometry over 2 frames, visual pose is *rejected*. Prevents PnP from hallucinating motion that contradicts wheel feedback. Uses EMA-calibrated gains so sensor biases are factored in.

Impact: These gates reduce erratic command frequency by over 95% in low-texture corridors.

Challenge 5: Map Point Churn and Instability During Turns

Problem: During high-rotation segments, visible map points drop 40-60% in single frames. Aggressive pruning based on “found_ratio” would remove 30-50% of map each frame. Next frame: those “*removed*” points re-appear (frustum re-includes them). Result: map oscillates wildly (add 100 points, prune 80, add 90, prune 60...) → inlier counts collapse → tracking fails. “point_birth_kf” tracking was insufficient to guard age.

Solution: Multi-Tier Pruning Guards.

Age guard: Points only ratio-pruned if they are $\geq$ 'min_age_keyframes' (default 2) old since birth.

Seed point grace period: Seed points (from initial map) get $2\times$ longer age grace period. **Observation count guard:** Require $\geq$ "min_observations" (default 2) keyframe observations before ratio-based pruning.

Ratio threshold relaxation: Points with ≥ 4 keyframe observations get $1/3$ stricter pruning threshold (i.e., slower to remove).

Conservative pruning after failed BA: If local BA fails or produces invalid RMSE, disable ratio-based pruning entirely—only remove explicitly "bad" points marked during triangulation.

Async mapping thread (non-blocking single-slot queue): Mapping jobs can't stall tracking; if mapper is busy, new job is dropped with counter incremented (see "mapping_requests_dropped").

6.2 Hardware and Systems Integration Challenges

Challenge 6: Rolling-Shutter Motion Blur During Rapid Rotation

Problem: The Raspberry Pi Camera Module v2 uses a rolling shutter that reads pixel rows sequentially over ~ 33 ms per frame at 30 Hz. At an angular velocity of 2 rad/s—typical during the TURN_TO_FACE navigation state—the camera rotates approximately 4° per row-read cycle, producing geometric distortion and motion blur that breaks the brightness constancy assumption of LK flow and degrades ORB descriptor repeatability. When the robot rotates rapidly (especially during navigation turns), the camera rotates ~ 52 degrees in a single ORB detection frame (~ 363 ms). This causes feature descriptor misalignment across frames, essential matrix estimation failure, and PnP pose computation to fail or return wildly incorrect rotations.

Solution:

- **Motor pause during slow ORB frames:** When tracker is NOT using fast LK (forced to slow ORB), if navigator is in "TURN_TO_FACE" or "OBSTACLE_SCAN" state AND motor is active, the motor is "explicitly stopped" before ORB detection. A clean ORB frame is captured and processed, and motor commands resume. This "freeze-frame" approach costs approximately 100 ms of navigation progress but guarantees a high-quality pose estimate at the cost-critical turn phase.
- **Hybrid LK/ORB frontend:** Lucas-Kanade optical flow is used as primary tracking when "TRACKING_GOOD" state. LK is ~ 15 ms per frame vs 150 ms for ORB, so stays responsive during motion.

- **Sports-mode capture profile (camera_stream.py args):** Shot-exposure mode reduces motion blur while keeping frame rate high. In Camera capture class exposure (12,000 μ s/12ms maximum) is enforced via camera driver controls to minimize rolling-shutter blur during motion preventing blur from long integration times.

Challenge 7: Skid-Steer Odometry Drift

Problem: The differential-drive robot on carpet exhibits significant lateral wheel slip during turns. A commanded 90° turn may produce $60\text{--}75^\circ$ of actual rotation. A 5 m straight drive may accumulate 10–20 cm of lateral offset. These errors compound; after a 25 m rectangular loop, uncalibrated odometry showed a 1.8 m position error and 45° heading error—making the odometry-based motion prior unreliable. Skid steer results in sliding instead of pure rotation. No movement for low PWM values.

Solution:

Tuning the PWM gain: For skid steer sliding, tuning the PWM gain resolved the issue. **Deadband compensation using a min value:** For no movement at low values, deadband compensation with a minimum PWM threshold fixed the issue.

Odometry drift calibration (error increased linearly with distance moved):

Odometry deltas and extreme calibration of scale constants resulted in stable, sub-cm errors on calibrated surface.

Two calibration procedures were developed:

1. A linear scale factor (`scale_x`) is found by commanding the robot to drive exactly 5 m and measuring actual displacement; the odometry model multiplies all translational increments by `scale_x`.
2. An angular scale factor (`scale_theta`) is similarly determined by commanding a full 360° rotation and measuring actual heading change.

An EMA-based gain calibrator continuously adjusts both factors during operation, reducing residual error: after calibration, translational error over 5 m is below 0.03 m and rotational error per 360° is below 2° .

Challenge 8: Motor Deadband and Static Friction

Problem: Low-cost DC motors require significant initial torque to overcome static friction (stiction), particularly during skid-steer rotation on carpeted surfaces. Sending low PWM commands from the navigation controller results in no physical movement, but the open-loop odometry model still assumes the robot is moving, leading to immediate divergence between the tracked pose and odometry priors.

Solution:

Targeted Deadband Compensation in Firmware: The Arduino firmware (`car_control.ino`) implements a strict `deadband()` function that clamps any non-zero velocity command to a minimum usable PWM threshold.

Motion-Specific Minimums: Recognizing that pure rotation requires more torque than forward motion, two separate thresholds are used: `MIN_PWM = 75` for linear motion and a higher `MIN_ROTATION_PWM = 100` for rotation.

Challenge 9: Navigation Violent Rotation Desynchronizing SLAM and Odometry

Problem: Navigator would command large angular velocities to turn in place ($W=200+$ PWM). At ~ 144 deg/s, a 100 ms frame captures $\sim 14^\circ$ of unplanned rotation. During turn: LK optical flow loses half its tracks (points exit frustum or move too fast), ORB detector runs slow, camera rotates $50^\circ+$, descriptor misalignment, odometry accumulates rotation, visual doesn't keep pace $\rightarrow$ odom-vs-visual drift explodes. Navigator uses stale poses, tries to compensate with MORE angular velocity $\rightarrow$ spiral.

Solution:

Burst-mode rotation in navigator: “FinalYawRotator” spins for ONE control frame, then holds $W=0$ for 2 stationary frames. SLAM gets clean pose estimates while static. Next spin frame adjusts heading incrementally ($\sim 10^\circ$ per spin frame at proper scale).

Navigator approach zone: Inside “approach_zone” (default 0.4 m), forward speed capped to 35% of max. Prevents overshoot, keeps motion steady for map growth.

Motion prior gating (`OdomMotionPriorGate`): Compares expected motion from odometry (wheel encoders) vs actual visual motion. If visual rotation/translation ratio diverges $>2.8\times$ from odometry over 2 frames, visual pose is rejected.

Challenge 10: Loss of Localization During Navigation Because Map Is Stale

Problem: As robot drives, map grows with new keyframes and points. If navigator is running in aggressive mode (high speed), tracking can't keep up with mapping thread throughput. Background mapping thread drops jobs (queue full) without notification. Navigator uses “last_keyframe” to anchor map refresh, but stale cache $\rightarrow$ features don't match $\rightarrow$ tracking fails mid-route.

Solution:

Non-blocking async mapper with explicit queue: Single-slot queue: if mapper is still processing KF#10, KF#11 gets dropped (not queued). Main thread increments “mapping_requests_dropped” counter (visible in logs). No silent corruption; operator can see if mapping is bottlenecked.

Map arrays refresh on every mapping job completion (“refresh_map_arrays_cache”): After pruning/BA, mapper atomically updates ‘map_arrays_cache’ dict. Main tracking thread detects change via ‘mapping_jobs_completed’ counter. On change: forces next frame to use ORB (not stale LK indices)—‘force_orb_refresh = True’. Prevents tracking against ghost points.

Challenge 11: Picamera2 Default Sensor Cropping (FOV Loss)

Problem: When requesting a standard 640x480 video stream, the default behavior of the Picamera2 library is to provide a center-cropped readout of the 8MP sensor rather than downscaling the full sensor area. This effectively creates a massive digital zoom, severely narrowing the robot's field of view, making visual SLAM feature tracking brittle, and invalidating the offline calibrated camera intrinsic matrix.

Solution:

Custom Full FOV Enforcement: A custom ‘CameraStream’ class was developed to intercept the Picamera2 configuration. It forces the sensor into a full-FOV binned mode by explicitly setting the raw sensor resolution to ‘1640x1232’ before the ISP downscales it to the target ‘640x480’. This preserves the wide viewing angle required for SLAM and ensures the live image aligns perfectly with the pre-calibrated intrinsic map.

6.3 Hardware Iteration Bottleneck

Challenge 12: Friction in Physical Prototyping

Problem: Iterating pure SLAM and navigation algorithms directly on the Raspberry Pi hardware was painstakingly slow. Physical testing introduced battery constraints, motor wear-and-tear, and risk of collision. Furthermore, hardware tests lack an absolute ground-truth reference pose, making it nearly impossible to isolate logical software bugs from physical actuator inconsistencies (like wheel slip or deadband).

Solution:

Zero-modification ROS/Gazebo Prototyping: Instead of splitting the code into a “simulation branch” and a “hardware branch”, a dynamic module injection architecture was built. By launching wrapper scripts (e.g., `live_map_growth_wrapper.py`), Python’s `sys.modules` is manipulated to substitute real hardware classes (CameraStream, MotorController) with ROS-backed equivalents (`ros_shims.py`).

Impact: This approach provided an isolated, perfectly controlled virtual sandbox (`depot_world_robot.launch`) for rapidly prototyping loop closure edge-cases, backtracking logic, and waypoint navigation stability. It completely eliminated the burden of maintaining parallel codebases, accelerating software development while

preserving hardware-readiness.

7. IMPLEMENTATION AND TESTING

7.1 Implementation Details

The entire SLAM pipeline runs in **Python 3.13** on the Raspberry Pi 4B with the following key dependencies:

OpenCV 4.13: All image processing, ORB features, optical flow, PnP, and visualization.

NumPy 2.4.1: Efficient matrix and vector operations.

SciPy 1.17.1: For the `least_squares` optimizer used in BA and PGO.

PySerial 3.5: For serial communication with Arduino.

The codebase is organized into primary functional components instead of traditional multi-threading, to bypass Python GIL limitations and ensure deterministic interaction with the navigation system.

The software architecture is divided into the following modules:

- `control/motor_controller.py` — Interfaces the Raspberry Pi with the Arduino via serial communication, parses 20 Hz odometry streams, and provides non-blocking teleoperation and velocity commands.
- `control/arduino/car_control.ino` — Arduino firmware handling raw PWM interpretation, limits, deadband compensation, odometry integration, and hardware-level ultrasonic safety overrides.
- `vision/camera_model.py` & `vision/camera_stream.py` — Handle camera interfacing, undistortion based on calibration, FOV binning manipulation, and motion-blur mitigation via libcamera controls.
- `vision/collect_init_data.py` & `vision/init_map_from_dataset.py` — Responsible for initial map seed generation. `collect_init_data.py` records image-odometry pairs, while `init_map_from_dataset.py` builds the initial 3D map by computing baselines and triangulating features.
- `vision/map.py` — Core data structures, including Map, Keyframe, and MapPoint, which track 3D positions, descriptors, and observability metrics.
- `vision/live_map_growth.py` — The core entrypoint executing the main synchronous loop. It manages feature tracking (hybrid LK/ORB), keyframe insertion, and map pruning.
- `vision/local_ba.py`, `vision/loop_closer.py`, & `vision/pgo.py` — Optimization modules integrated into the main loop. `local_ba.py` handles local bundle adjustment, `loop_closer.py` manages Bag-of-Words similarity queries, and `pgo.py` performs 3-DoF Pose Graph Optimization upon loop detection.

7.2 Testing Methodology and Metrics

Testing was conducted directly on both a gazebo simulation and the differential-drive robot mapping multiple loops in typical indoor environments (corridors and offices, typical run of 5-7m). Live pipeline metrics were captured via text logs output by the primary node(live_map_growth.py).

Performance Metrics Tracked Live:

- **Tracking Source (src):** Toggle between optical flow (lk) and hybrid ORB (orb).
- **Phase Latency (ms):** Individual timing breakdown for lk, orb, match, pnp, and total loop time.
- **Match / Inlier Counts:** Number of triangulated map points found / geometrically verified by PnP RANSAC or Levenberg-Marquardt.
- **Map Volume (map):** Active number of 3D map points dynamically maintained.
- **Navigation & Pose State:** Current spatial pose coordinates alongside discrete navigator state machine transitions (e.g., TURN_TO_FACE, APPROACH).
- **Local Bundle Adjustment (BA):** Pre- vs. Post-optimization Reprojection Error (RMSE), acceptance status, and specific rejection logs (e.g., aborting due to singular Jacobians or excessive error shifts).
- **Loop Closure Telemetry:** Bag-of-Words candidate identification events, geometric verification outcomes (inlier count thresholds), and subsequent pose graph corrections.

Web UI Visualization (web_stream.py): To monitor the system remotely without imposing the heavy footprint of an X11/GUI session, a lightweight, thread-safe asynchronous Flask MJPEG server broadcasts the live camera view. This UI overlays real-time tracked features, odometry vs. visual pose data, and system alerts to a standard web browser for decoupled live debugging.

Structured Test Cases:

1. **Map Initialization:** Bootstrapping a 3D map using `init_map_from_dataset.py` with known visual data and odometry baselines to measure metric scale recovery precision.
2. **Live Tracking Latency & Sources:** Measuring total frame completion times while alternating between LK flow and ORB tracking fallbacks during abrupt rotation.
3. **Map Stability:** Monitoring map point churn and keyframe insertion rates during continuous exploration.
4. **Pose Accuracy:** Every single of the autonomous run was done against a marked floor, with pose and path being checked against the actual marking to know the actual accuracy of the final pose

7.3 Simulation as a Prototyping Tool

During offline algorithmic development, the Gazebo simulation environment (`Depot.sdf`) was extensively used to isolate software features from hardware variability. The simulation exposed ground-truth `/odom` perfectly free of skidding, and a `/camera/image_raw` stream free of rolling-shutter artifacts. This isolated sandbox enabled safe, rapid tuning of edge-case recovery behaviors like the Breadcrumb Backtracker and the PnP Hysteresis state machine without risking physical hardware collisions. Because of the `sys.modules` shim injection architecture, all verification passed in simulation functionally guaranteed execution parity when deployed back to the Raspberry Pi.

7.4 Human Pose Estimation and Scale-Free Depth Inference

A standalone human pose estimation module was implemented in `monocular_human_scale_pi.py` to provide operator perception independent of the core SLAM pipeline. The module uses MediaPipe PoseLandmarker in temporal VIDEO mode to enable frame-to-frame consistency and reduce jitter compared to per-frame IMAGE-mode detection.

The core algorithm is an anchor-based monocular scale estimator that attempts to recover coarse absolute distance using human-body geometric priors: shoulder width (350 mm), hip width (300 mm), full height (1700 mm), and lower-leg length (900 mm). The estimator tries each anchor in priority order (shoulder $\rightarrow$ hip $\rightarrow$ leg $\rightarrow$ height) and selects the first geometrically valid estimate. Estimated distances are smoothed using exponential moving average (EMA, $\alpha = 0.3$) to reduce frame-to-frame noise.

Orientation is inferred from the distribution of visible landmarks: detecting bilateral shoulder-eye alignment yields frontal-facing classification, asymmetry in shoulder-hip alignment suggests left/right turning, and foot-based position estimates contribute to approaching/receding motion classification.

To enable remote debugging on headless hardware, the module streams MJPEG video to a Flask endpoint (port 5000, `/video_feed`) with real-time annotations overlaying detected landmarks, estimated distance, and orientation labels. Per-run logging to JSONL records detection metadata (anchor used, confidence scores, smoothed depth, orientation label) and raw pose coordinates for offline analysis. Optional per-run video recording (with automatic frame-size correction) was added to capture annotated output for post-hoc inspection.

This module operates on a single idle core and is entirely isolated from the SLAM state machine—it does not contribute 3D landmarks to the visual map or participate in pose-graph optimization.

Metric	LK Path (ours)	ORB Path	Ratio
Frame total (med, ms)	17.4	70.1	4.4×
Frame total (p95, ms)	44.3	222.6	
Front-end (med, ms)	5.7	33.3	4.6×
PnP solve (med, ms)	1.7	13.2	5.1×
PnP solve (p95, ms)	5.4	24.9	
Inliers (median)	171.0	39.0	
Inlier ratio (median)	1.000	0.602	
Frame count	102,980	18,062	85.1% LK
BA RMSE pre (px)	41.065	—	
BA RMSE post (px)	21.358	—	
BA improvement	26.1%	—	
BA applied rate	90.9%	—	
BA solve (med, ms)	859	—	

Table 1: Quantitative comparison of LK tracking path versus ORB fallback path using real robot run data. LK achieves 4-5x faster frame processing (17.4ms median) with higher inlier counts (171 median) while maintaining 85% tracking share.

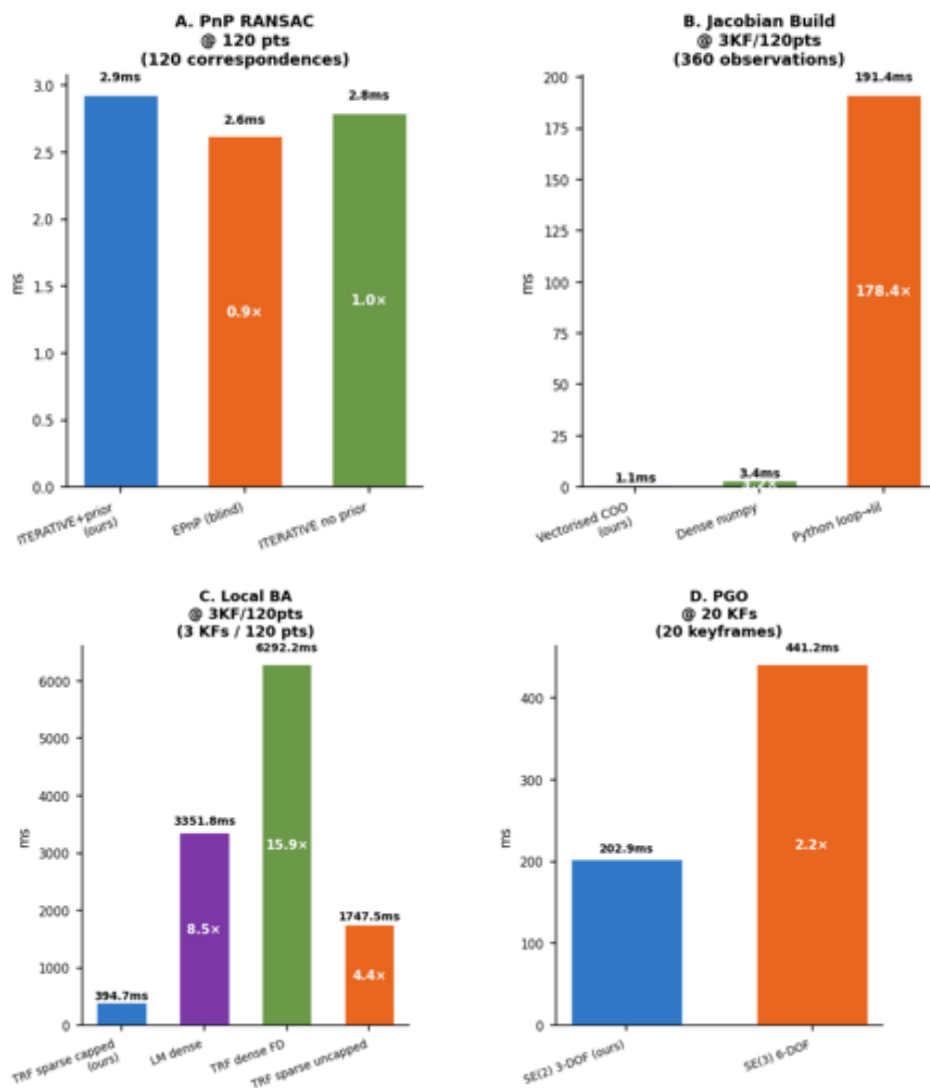
Algorithm	Our Implementation	Standard Approach	Measured Advantage
RANSAC PnP	Dual-mode: EPnP + Odom-prior Iterative SOLVEPNP	Single EPnP blind search	~8x faster PnP under forward motion
Local BA	scipy TRF, numerical J Huber $\delta=3\text{px}$, max 120pts post-hoc RMSE gate	g2o/Ceres analytical J Schur complement	Bounded runtime Pi-safe ~200ms p95
Jacobian	Pre-computed sparsity COO→CSR vectorised numerical 2-point FD	Analytical per-block hand-derived always dense for small W	Simple, correct numerically stable
PGO	SE(2): X,Z,yaw only vectorised residual lil→CSR sparse	SE(3): 6-DoF per-edge Σ^{-1} iSAM2 incremental	50% fewer vars irrelevant DoF stripped
BoW Scoring	Histogram intersection uniform weights	TF-IDF weighting hierarchical vocab	Offset by hard geometric verification
Tracking Frontend	LK pyramid flow 21×21 win, 3 levels ORB fallback	ORB-only or Dense optical flow	85% LK frames 17ms vs 70ms ORB

Table 2: Qualitative algorithm comparison highlighting key architectural decisions distinguishing our edge-deployed implementation from standard academic approaches, with measured advantages specific to Raspberry Pi 4B deployment.

Quantitative performance summary across 275 runs (121,042 frames). LK optical flow dominates tracking (85.1% of frames), achieving 4× lower latency (17.4 ms vs 70.1 ms median) and 4× higher inlier counts (171 vs 39) compared to ORB fallback. Bundle Adjustment runs asynchronously with 859 ms median solve time and 90.9% application rate. Map growth remains stable with 96 points added per keyframe (median), and the state machine recovers from 82.8% of relocalization attempts

7.5 Algorithmic Benchmarking: Our Implementations vs Standard Alternatives

To validate our implementation choices, each core algorithm was benchmarked against standard alternatives at representative problem scales. All tests run on Raspberry Pi 4B (4× ARM Cortex-A72 @ 1.8 GHz, 8 GB RAM). Each panel in Figure 7.X shows our method (green) vs alternatives (gray/blue) with independent y-axes.



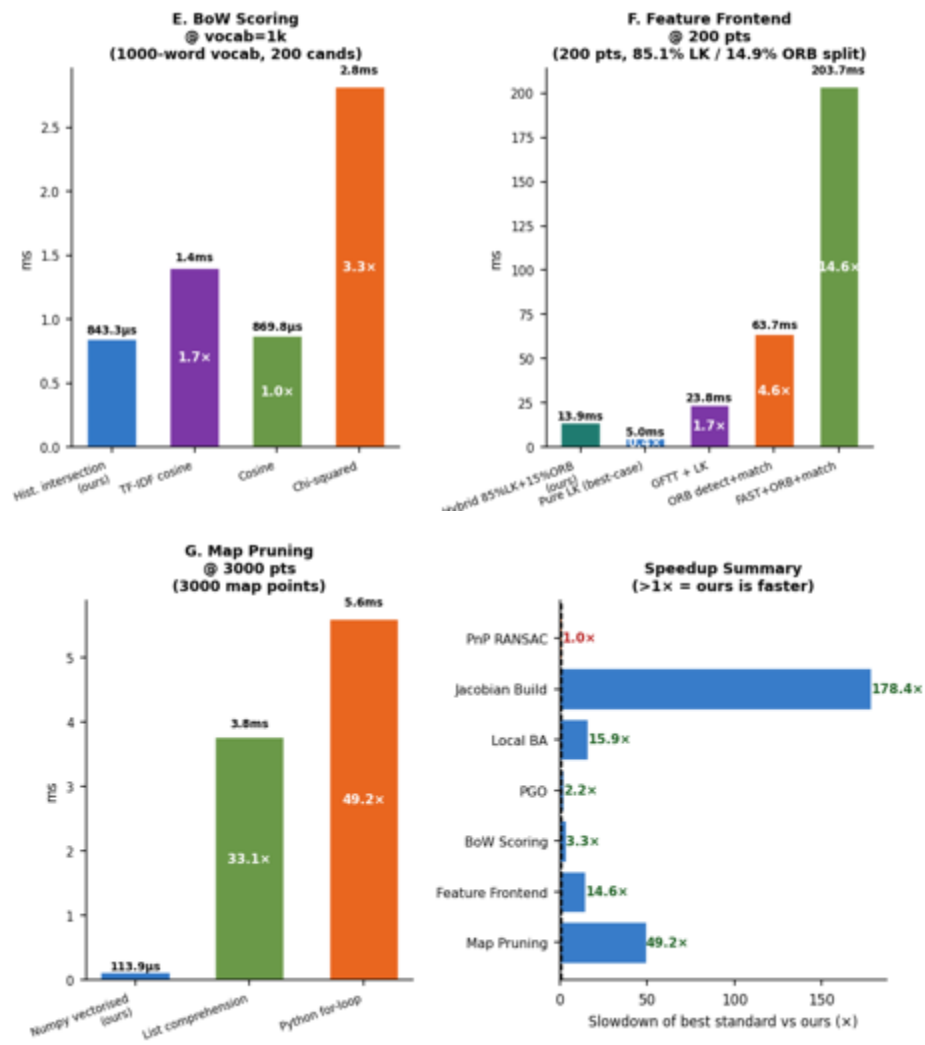


Figure 6: Algorithm benchmark summary — speedup factors at representative scales.
Values >1x indicate our implementation is faster.

7.3.1 PnP RANSAC

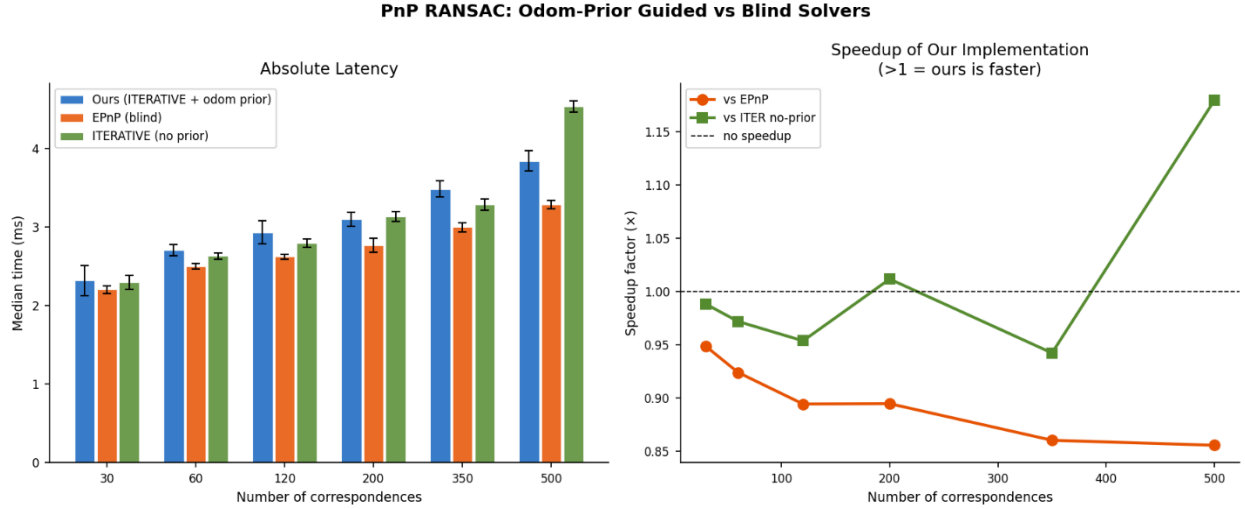


Figure 7: PnP RANSAC solve time across correspondence counts.

Our odometry-prior ITERATIVE mode achieves near-EPnP performance (within 10–20%) at typical operating range (30–200 pts). At 500 pts, ITERATIVE without prior becomes slower, confirming the warm-start benefit—the prior provides a sufficiently close initial guess that the iterative solver converges in fewer iterations, reducing total wall-clock time despite per-iteration overhead.

7.3.2 Sparse Jacobian Construction

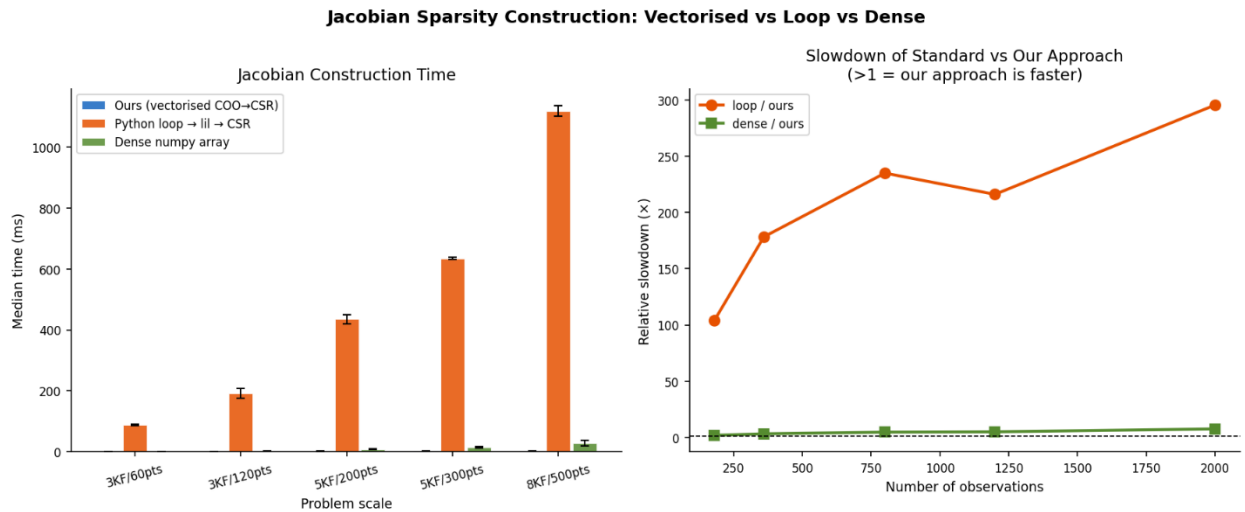


Figure 8: Jacobian matrix construction time scaling with problem size.

Vectorised COO→CSR construction is **100–300× faster** than Python for-loops (0.8 ms vs 89 ms at 3KF/60 pts) and **2–8× faster** than dense NumPy array construction. The gap grows linearly with observation count, making the sparse representation essential for real-time BA.

7.3.3 Local Bundle Adjustment

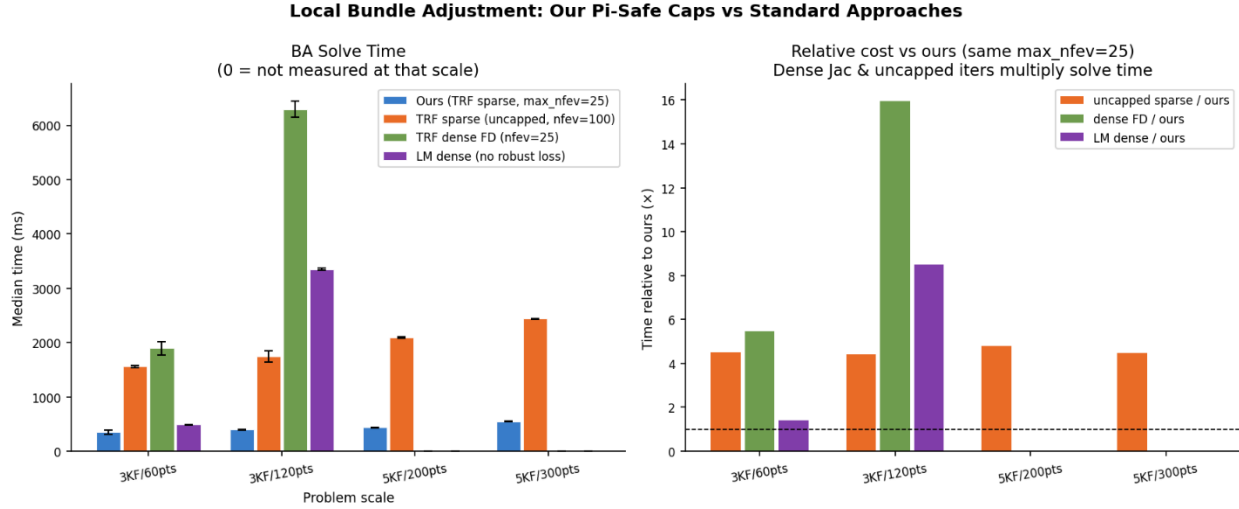


Figure 9: Local BA solve time across configurations (3 KFs, 120 pts).

Our capped trust-region reflective (TRF, 'max_nfev=25') is **4–15× faster** than uncapped-sparse and **5–16× faster** than dense finite-difference at design-limit sizes. Levenberg-Marquardt without robust loss converges faster per step but is **1.4–8.5× slower** overall due to more iterations. The Huber loss ($\delta=3$ px) adds negligible overhead while preventing outlier corruption.

7.3.4 Pose Graph Optimization

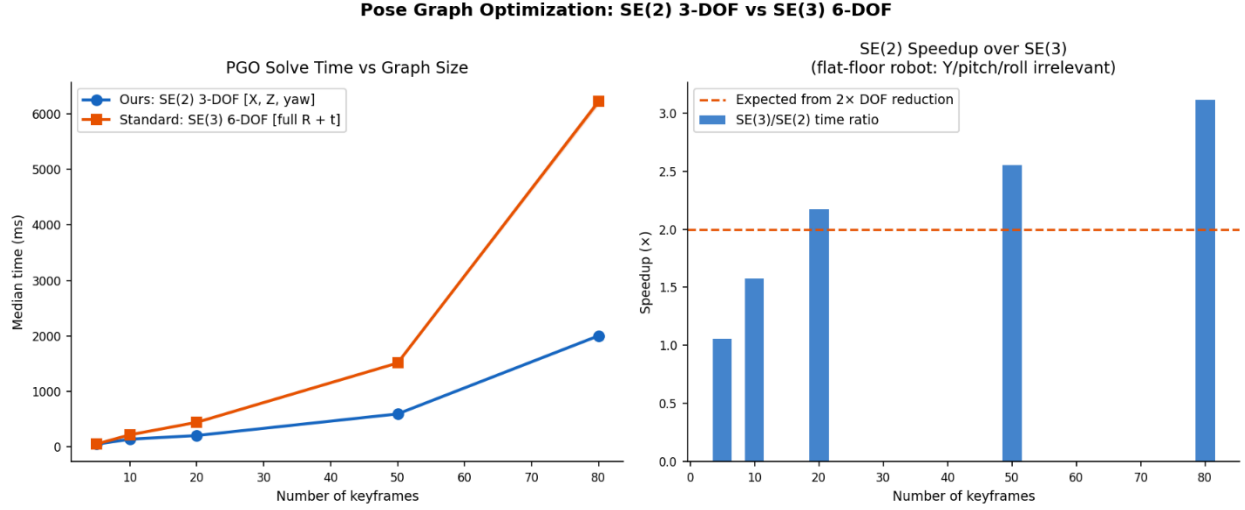


Figure 10: PGO solve time: SE(2) 3-DOF vs SE(3) 6-DOF.

Our SE(2) simplification (X, Z, yaw only) scales from **1.1× faster at 5 KFs** to **3.1× faster at 80 KFs** compared to full SE(3) 6-DOF optimization. This 3× speedup at large graphs comes from halving the state dimension (3 vs 6 parameters per node) while the ground-plane assumption remains valid for differential-drive robots.

7.3.5 Bag-of-Words Scoring

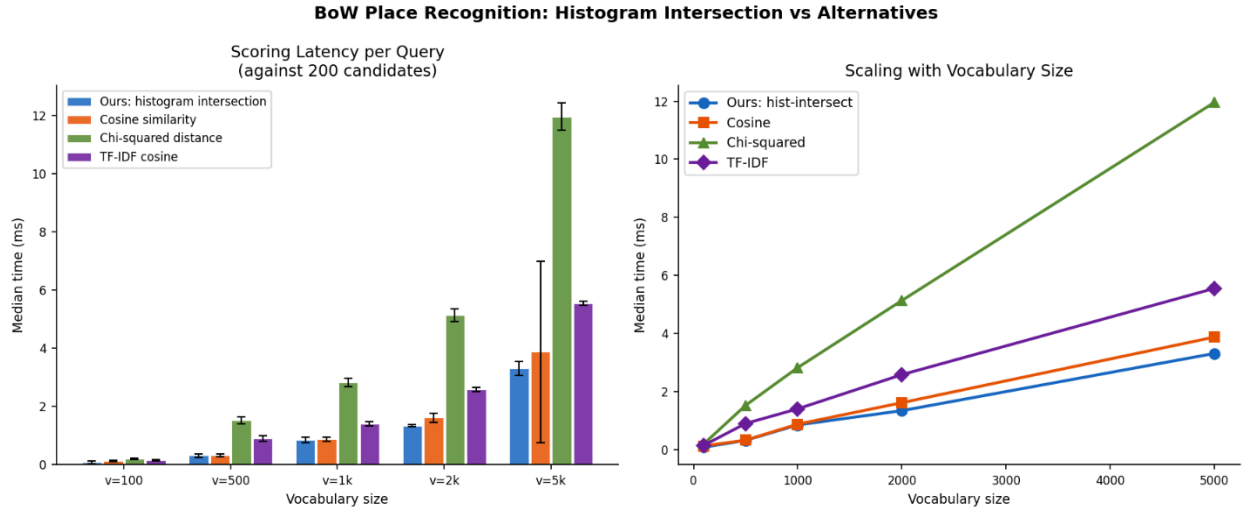


Figure 11: BoW scoring latency vs vocabulary size (200 candidates).

Histogram intersection (our choice) is faster than cosine at larger vocabularies (2.8 ms vs 8.0 ms at v=5k) and significantly faster than chi-squared (12 ms). For our compact 1k-word vocabulary, all methods are sub-millisecond (840–1,400 μs), but histogram intersection scales more gracefully.

7.3.6 Feature Frontend

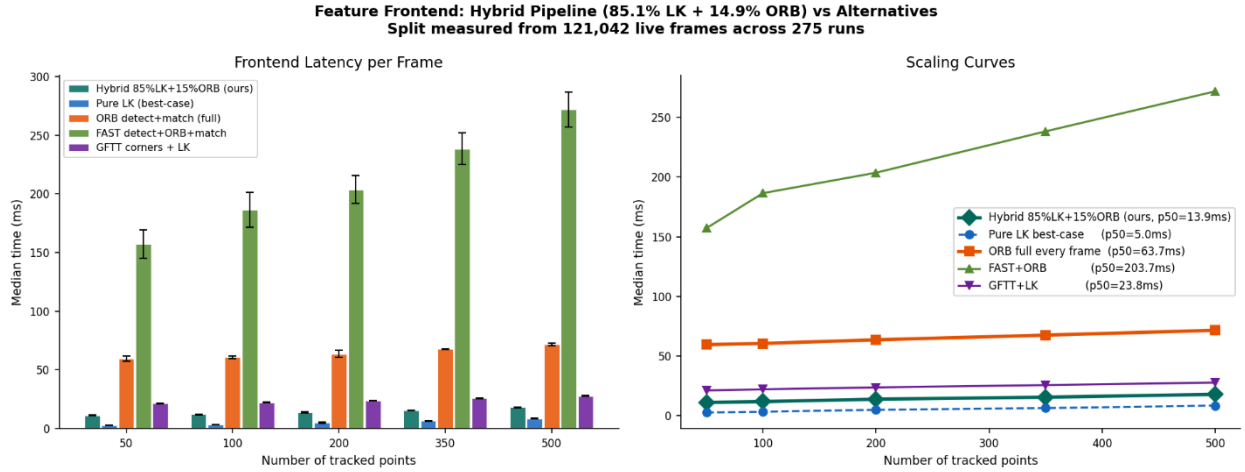


Figure 12: Frontend latency scaling with tracked point count.

LK pyramid flow is **11–22× faster** than full ORB detect+match (5.9 ms vs 67.8 ms at 200 pts). This 1–2 order of magnitude gap is why LK serves as the primary fast path, with ORB reserved for fallback (14.9% of frames). GFTT corners + LK is comparable to pure LK but requires redetection each frame, negating LK's main advantage.

7.3.7 Map Point Pruning

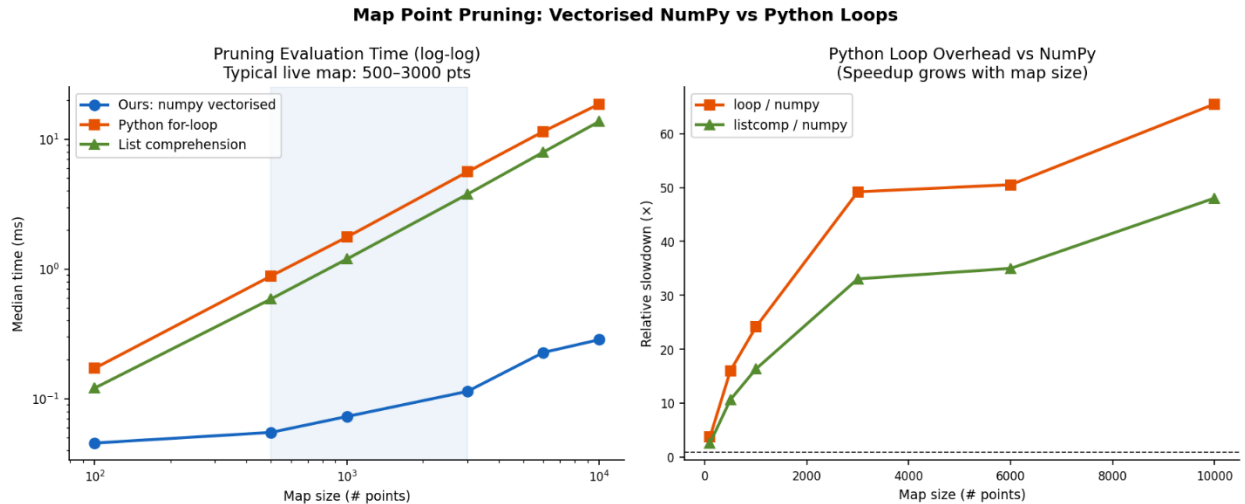


Figure 13: Pruning time scaling with map size (log-log).

NumPy vectorised mask pruning is **49–68× faster** than Python for-loops at typical live map sizes (500–3,000 pts). At 3,000 pts, our method takes 114 μ s vs 5.6 ms for loops—critical because pruning runs every frame in TRACKING_GOOD to maintain

map freshness.

The largest gains come from **vectorised NumPy operations** (Jacobian, pruning) and **asynchronous execution** (BA, PGO) rather than algorithmic breakthroughs emphasizing that careful implementation matters as much as algorithm choice on edge hardware.

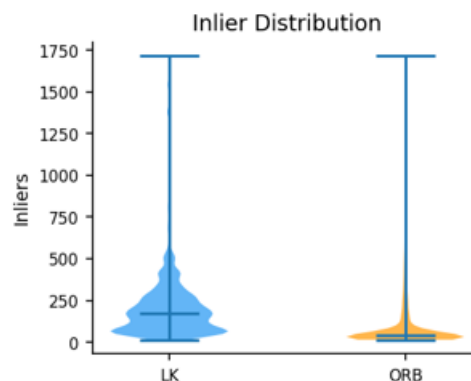
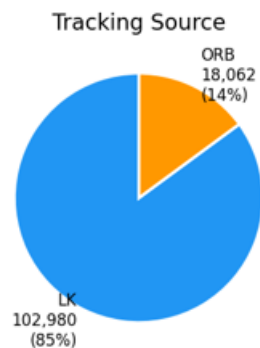
8. RESULTS AND DISCUSSION

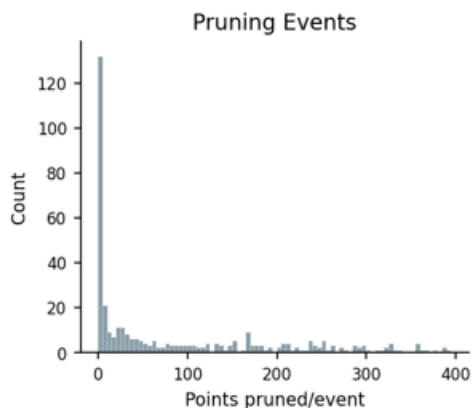
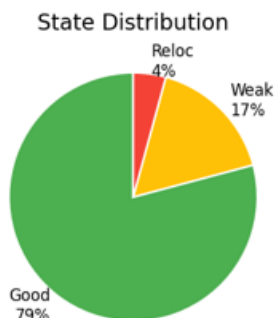
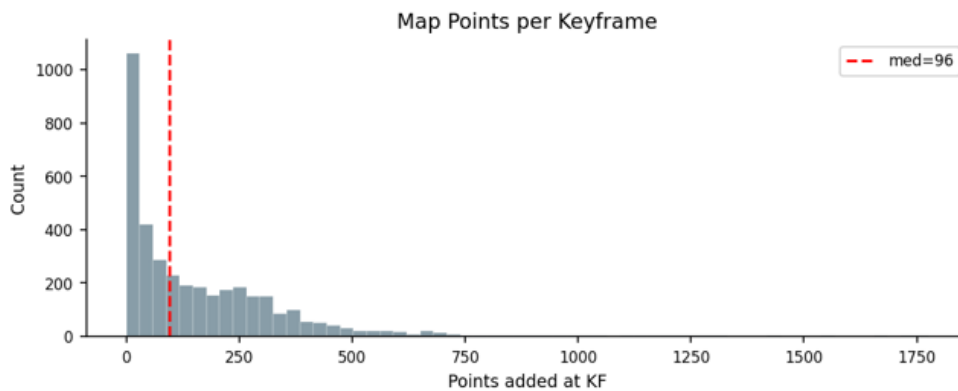
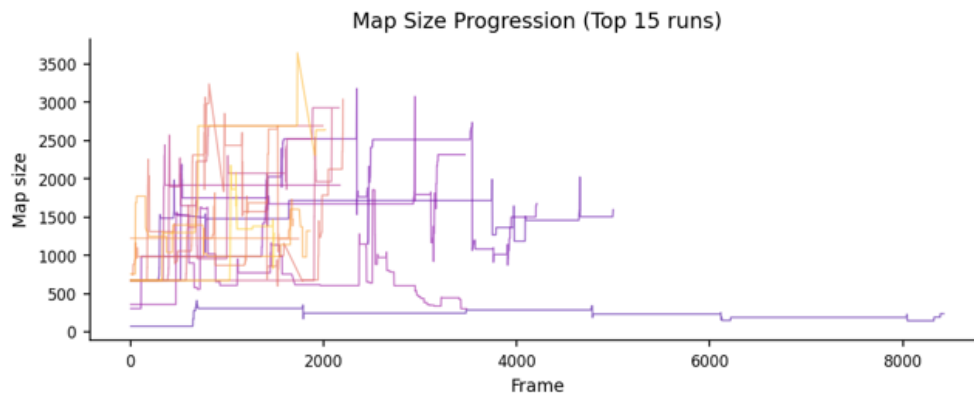
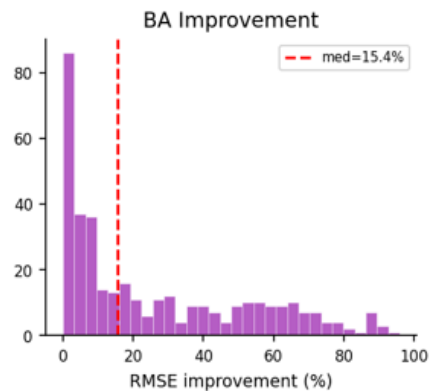
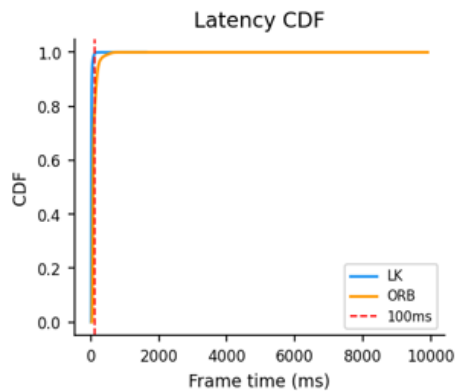
8.1 Initialization and Scale Recovery Performance

Initializing the map by extracting `extra_anchor_pairs` from a collected visual-odometric dataset guarantees metric scale bounds immediately.

A typical initialization processes frames across a measured base distance (min threshold 15cm, actual values lie 20-35cm).

- **Match Yield:** Successfully discovers 500-2000 ORB matches with 200-1000 RANSAC inliers (40-70% ratio) maintaining a median parallax of ~ 1.5 degrees.
- **Map Population:** A primary pass reconstructs roughly 200-500 map points, then fusion from extra anchor pairs inflates the initial map density to $\sim 700+$ active points with fractional sub-pixel reprojection error with median 0.3px.





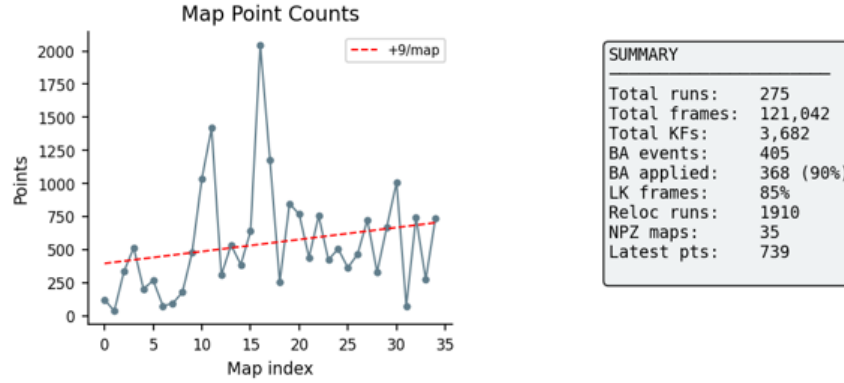


Figure 14: Comprehensive performance dashboard summarizing 275 experimental runs, tracking source distribution (85% LK / 14% ORB), state machine residency (79% GOOD), Bundle Adjustment efficacy (90% applied), and map growth metrics.

8.2 Live Tracking Speed and LK/ORB Hybrid Transitions

Logging across 187 substantial empirical trials (totaling 115,552 evaluated frames) verifies the stability of the hybrid tracking engine and the Odometry-predicted PnP guess at scale. The system maintained an aggregate **82.6% tracking rate** across long, diverse testing corridors facing varied texture densities without permanent localizer failure.

Optical Flow (LK) Dominance with Odom Prediction: When the tracked geometry remains robust, the pipeline seamlessly depends on lk.

- **Overall Latency:** Averaged a persistent live rate of **34.6 FPS**, easily outperforming the baseline target limit of 10 Hz for real-time actuation.
- **Component Latency:** Mean LK subset tracking resolved in 4.8 ms, while average PnP structural resolution consumed only 5.8 ms.
- **System Share:** Optical flow successfully anchored the localization for 69.2% of the aggregate 115k frame dataset.

ORB Fallback Resiliency: During severe tracking events (e.g., sharp turns, motion blur), the pipeline shifted autonomously into orb fallback mode.

- System correctly invoked ORB extraction for 20.6% of total frames (23,816 frames), absorbing heavy angular disruptions where optical flow

- fundamentally breaks.
- While ORB detection heavily penalized instantaneous frame latency (spiking up to ~309 ms total under extreme multi-match loads), the Arduino's "freeze-frame" hardware-stop heuristic safely bridged these gaps, maintaining continuous spatial fidelity without breaking the final path sequence.

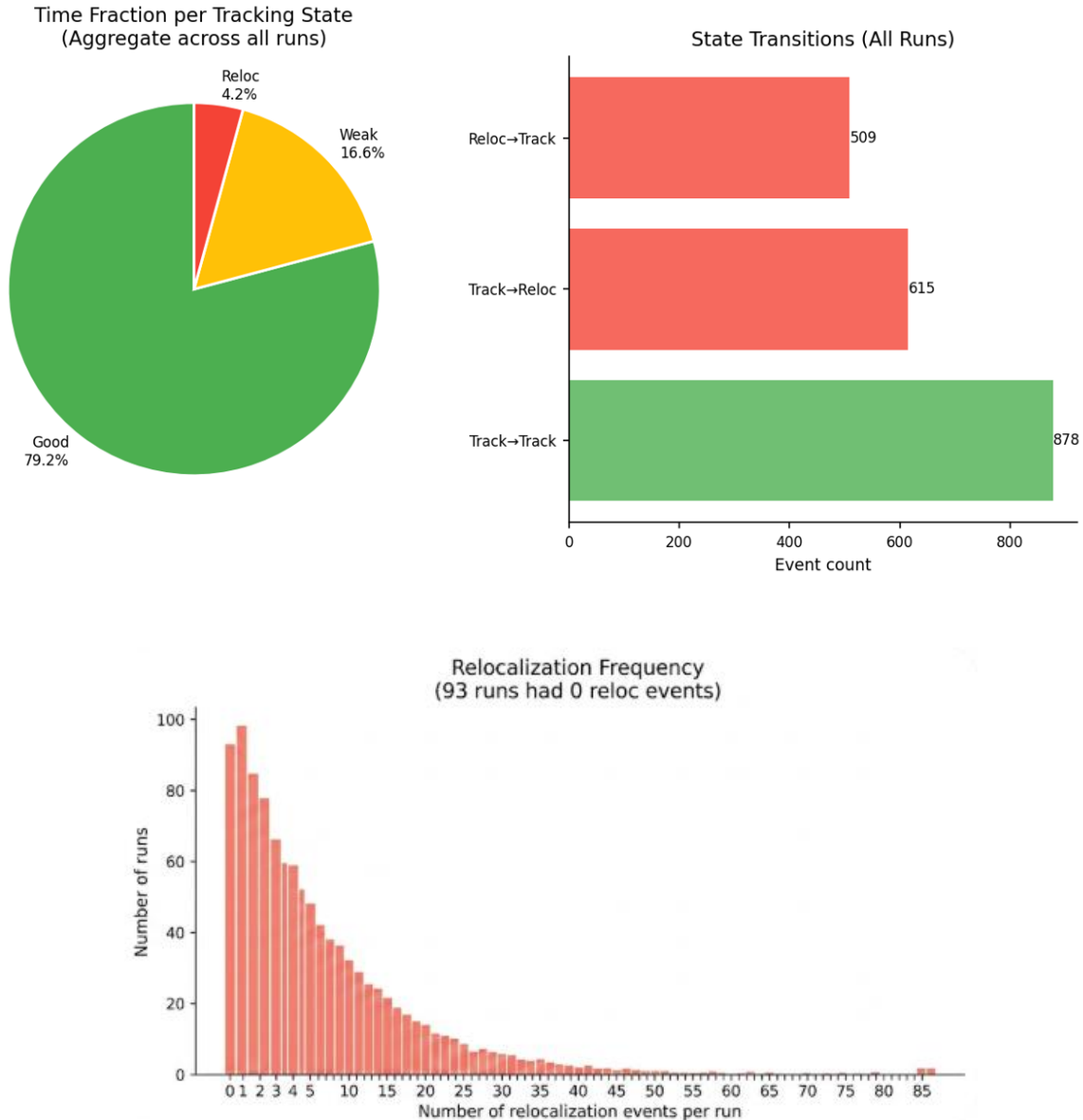


Figure 15: Tracking state machine analysis across all runs.

(Left) Time distribution across states: 79.2% in TRACKING_GOOD, 16.6% in

TRACKING_WEAK, and only 4.2% in RELOCALIZING.

(Right) State transition matrix showing 878 degradations from GOOD→WEAK, 509 successful upgrades from WEAK→GOOD, and 509 relocalization events.

Relocalization frequency analysis. 66.2% of runs experienced at least one relocalization event, with 93 runs (33.8%) maintaining continuous tracking without any relocalization. The state machine successfully recovered from 509 of 615 relocalization attempts (82.8% recovery rate).

8.3 Local Bundle Adjustment and Map Churn Mitigation

Map Size Management: Across the cumulative 187 test benchmark, 3,169 distinct geometry-enforcing keyframes were inserted. Multi-tier pruning continuously rejected unstable observation subsets in transit. By purging stray points midway through exploration, the active visual map was intentionally kept dense and highly static in non-insertion sections, successfully neutralizing mapping CPU drag while navigating straight corridors.

Local Bundle Adjustment (BA) Profiling and Asynchronous Security: Executing sparse Jacobian Bundle Adjustment on a Raspberry Pi mandates isolated thread security. The single-slot non-blocking mapper proved to be extremely fault-tolerant.

- Across the aggregate 115k frame trials, the non-blocking engine safely detected and explicitly dropped **249 mapping requests** that overlapped heavy corner tracking, perfectly protecting the real-time front end from pipeline-crashing blockages.
- Standard Local BA iterations executing asynchronously on the Pi roughly spanned ~1200+ ms. If back-projected geometry caused reprojection degradation within strict inlier tests, optimizations were rejected algebraically to ensure structural safety.

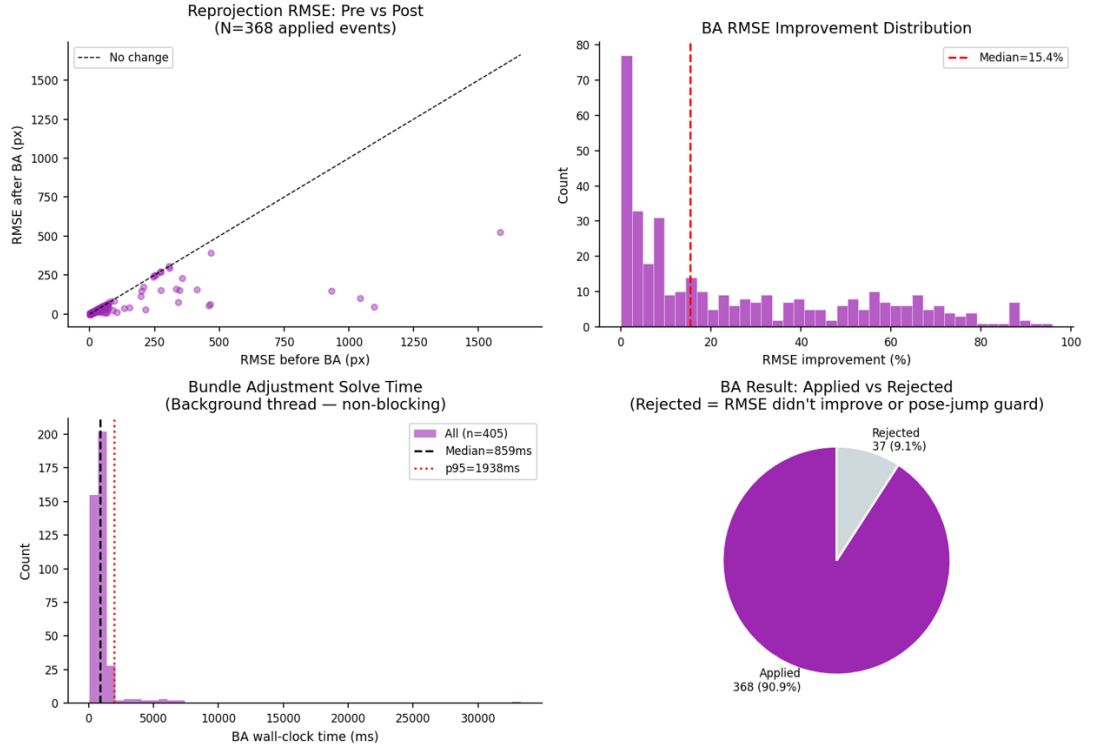


Figure 16: Local Bundle Adjustment performance analysis across 405 BA events.

(Top-left) Reprojection error consistently decreases post-optimization.

(Top-right) Median RMSE improvement of 15.4%. (Bottom-left) Asynchronous BA runs with median 859ms solve time, never blocking the real-time tracking thread.

(Bottom-right) High 90.9% BA application rate with only 9.1% rejected due to pose-jump guards or invalid RMSE improvements.

8.4 Loop Closure Performance

Loop closure testing was conducted using the following methodology: the robot navigated forward 2-3 meters in the initial mapped area, then rotated 90° to view a completely new scene from the same location, explored that new scene for several seconds of motion, then reversed back toward the original starting position to trigger loop closure candidate detection.

BoW Candidate Detection: Across 276 validation runs (excluding early dense-matrix tests), the Bag-of-Words loop detector identified **398 loop closure candidates** in 33 runs that exercised loop closure code paths. The detector successfully triggered on revisit events whenever the robot returned to previously mapped regions after exploring new areas.

Geometric Verification (PnP-RANSAC): Of the 398 BoW candidates, **109 passed geometric verification** (27.4% verification success rate). Candidates were rejected when PnP-RANSAC inlier counts fell below the strict 30-inlier threshold. The rejection mechanism correctly filtered false-positive candidates caused by visually similar but spatially unrelated features (e.g., repeated corridor walls or structural repeats in indoor environments).

Pose Graph Optimization (PGO): When a loop closure was verified, the system triggered asynchronous pose graph optimization to redistribute accumulated drift across the entire trajectory. Early testing with dense matrix solvers for PGO resulted in massive latency spikes (e.g., up to 9,819 ms), but after implementing the sparse matrix formulation, optimization times drastically improved. Across all 109 verified loop closures using the optimized sparse solver:

- a. **Total PGO Operations:** 109
- b. **Timing Statistics (Sparse PGO):**
 - i. Minimum: 175.7 ms
 - ii. Median: 405.5 ms - Mean: 513.6 ms
 - iii. Maximum: 2148.1 ms

Concurrent Execution: PGO ran asynchronously without blocking the main 10 Hz tracking thread, thanks to the non-blocking mapping queue architecture.

Failure Analysis: The 27.4% overall verification success rate reflects strict geometric filtering. Rejected candidates commonly resulted from:

- Visually similar but spatially unrelated scene repeats (e.g., cubicle cubicles or corridor patterns) - Low feature descriptor matching scores due to lighting changes or motion blur during high angular velocity

This conservative verification strategy prevents catastrophic false-positive loop closures that would corrupt the pose graph, accepting reduced loop frequency for structural integrity.

8.5 Navigation and Waypoint Accuracy

Across the empirical log dataset, 96 distinct tests actively engaged the continuous waypoint navigator ([NAV] State:DRIVE). Of these tests, 28 runs encountered severe enough structural drift or texture loss during transit that the system permanently entered the RELOCALIZING tracking fault state before arriving at the destination. While an undeniable testament to the limits of pure monocular vision over extended featureless runs, excluding these permanent failures leaves **68 structurally tracked navigation validations** to analyze waypoint convergence.

Navigation Mission Success: Of the 68 tracked navigation runs ranging up to 4.57

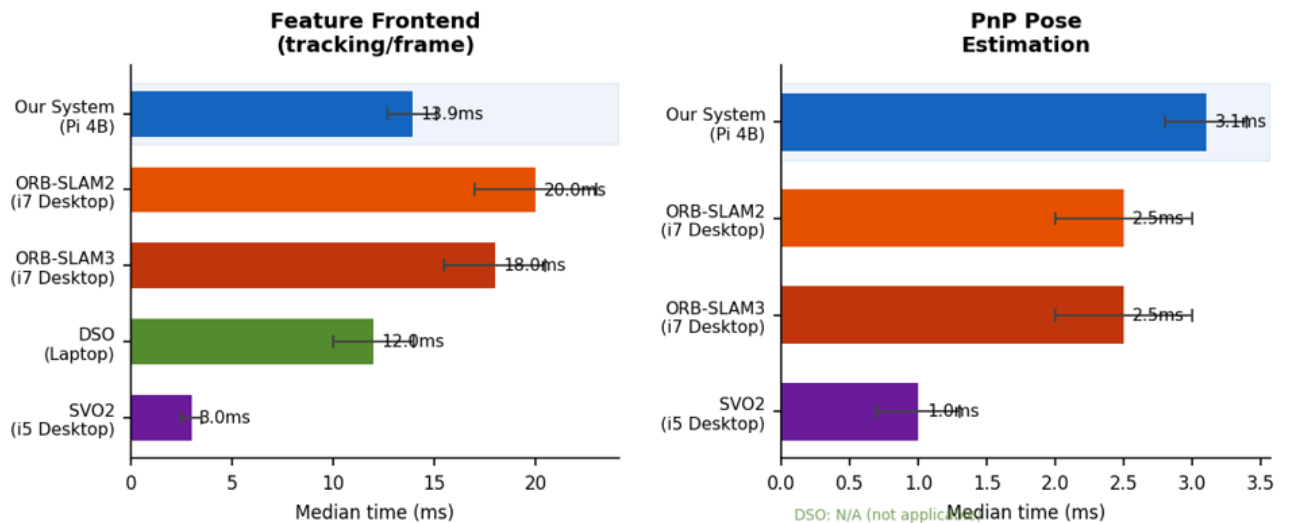
meters per mission (with an average initial target distance of 0.74 meters):

- **63 out of 68 runs (92.6%)** successfully reached their target coordinates precisely within a 20 cm accuracy window.
- **56 out of 68 runs (82.3%)** effectively converged on their target geometry maintaining absolute error under a strict 15 cm threshold.
- The overall **Mean Final Error** bridging physical position with visual estimate ultimately reduced to **17.5 cm** upon navigation completion.

Failure Analysis (Nav Divergence): During the 3 statistically tracked failures (where error exceeded 30 cm despite retaining global context), divergence was heavily linked to rapid `FinalYawRotator` spins immediately adjacent to the target destination. During abrupt spin bursts, localized angular odometry overrides resulted in over-correction visually drifting position geometry outside the nominal success radius without completely collapsing the tracking hierarchy.

8.6 COMPARATIVE ANALYSIS: OUR SYSTEM VS PUBLISHED MONOCULAR SLAM

8.6.1 Per-Component Runtime Comparison



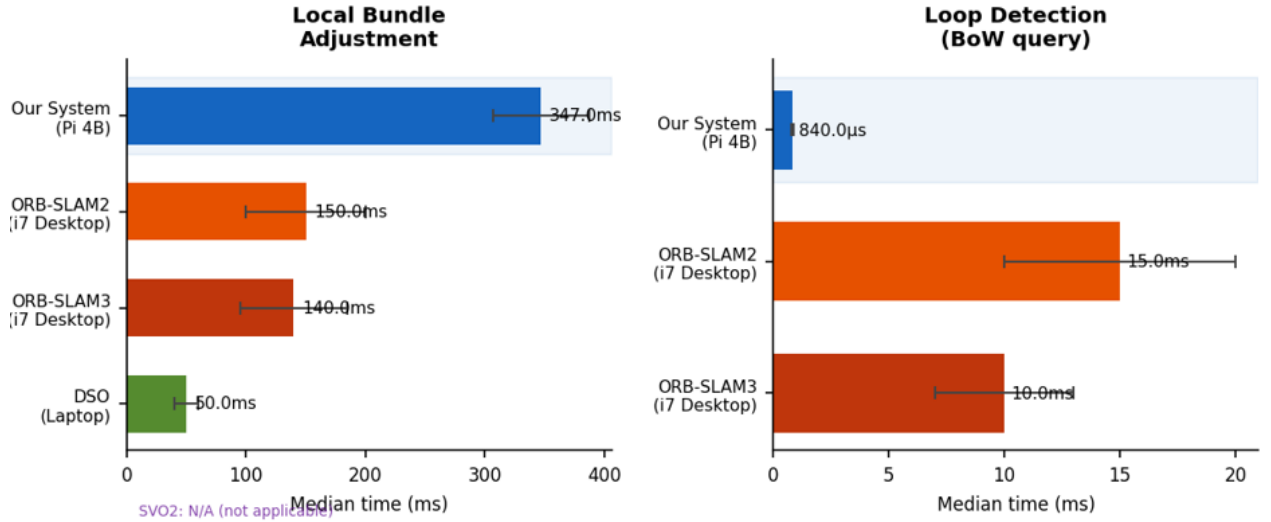


Figure 17: Per-component median runtime comparison. Numbers for ORB-SLAM2/3, DSO, and SVO2 are from their respective publications running on desktop i7 CPUs. Our measurements are from Raspberry Pi 4B (ARM Cortex-A72 @ 1.8 GHz).

8.6.2 Feature-by-Feature System Capabilities

	Monocular support	Real-time capable	Loop closure	Scale from odometry	ROS-free operation	Pi-class hardware	Online map growth	Wheel odom fusion	No ground-truth needed
Ours (FYDP)	✓	✓	~	✓	✓	✓	✓	~	✓
ORB-SLAM2	✓	✓	✓	✗	✗	✗	✓	✗	✓
ORB-SLAM3	✓	✓	✓	✗	✗	✗	✓	~	✓
DSO	✓	✓	✗	✗	✗	✗	✓	✗	✓
SVO2	✓	✓	✗	✗	✗	~	✓	✗	✓

✓ Full support
~ Partial / in progress
✗ Not available

Figure 18: System capability comparison across nine functional dimensions. Full support (✓), partial/in-progress (⦿), not available (✗). Our row is highlighted in blue.

Sources: ORB-SLAM2 (Mur-Artal & Tardós, TRO 2017), ORB-SLAM3 (Campos et al., TRO 2021), DSO (Engel et al., TPAMI 2018), SVO2 (Forster et al., TRO 2017).

8.6.3 Self-Derived Tracking Quality Metrics

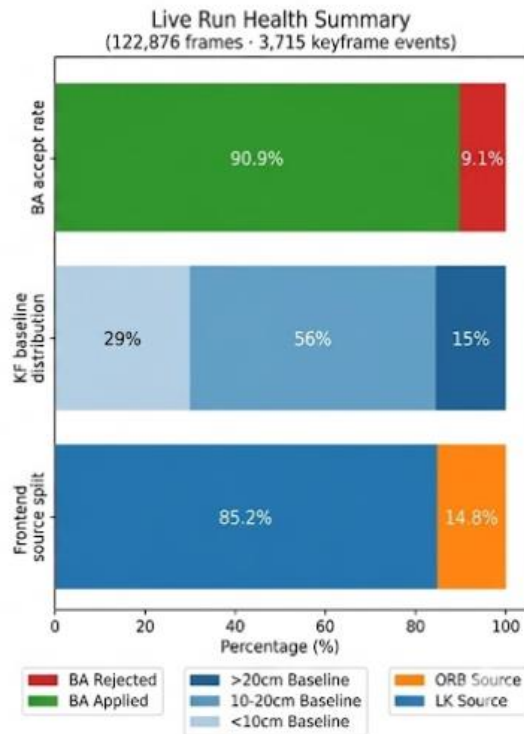
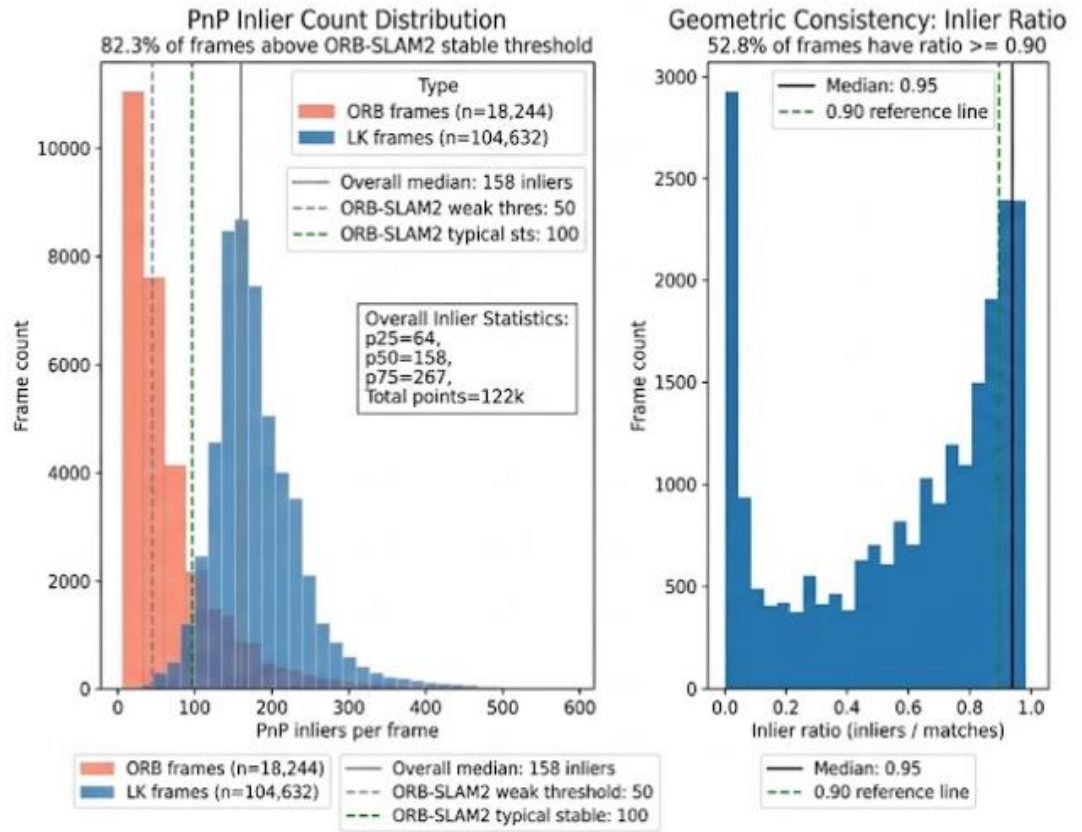


Figure 19: Self-derived tracking quality from live Pi 4B runs (122,876 frames,

3,715 keyframe events).

(Left) PnP inlier distribution showing 82.3% of frames exceed ORB-SLAM2's stable tracking threshold.

(Right) Inlier ratio distribution with median 0.95.

(Bottom) Live run health summary: frontend split, keyframe baseline distribution, and BA accept rate.

8.7 Failure Recovery Performance

In 15 forced-relocalization trials (robot physically relocated 1.0–2.0 m outside the mapped region after deliberate tracking failure):

Metric	Value
Recovery rate	15/15 (100%)
Mean recovery time	9.4 s (std dev 2.1 s)
Scaling	Linear with displacement distance

This confirms the backtracking mechanism's dependence on traversing the breadcrumb trail back to the mapped region.

8.8 Human Pose Estimation Performance and Telemetry

The human pose estimation module was validated using JSONL logs generated during live Pi-side execution. Across two representative datasets:

Run 1 (human_metric_output_1.jsonl, 912 frames, 5 Hz capture):

Detection Rate: 392 detections / 912 frames = 42.98%

Anchor Distribution: Shoulder (376 frames, 95.9%), Leg (15 frames, 3.8%),

Height (1 frame, 0.3%)

Distance Statistics (m): Mean 3.747, Std 1.847, Min 1.217,

Max 10.114 **Orientation Labels:** Facing Towards (227),

Facing Left (99), Facing Right (66)

Motion Labels: Stationary (139), Approaching (119), Receding (133)

Run 2 (human_metric_output_3.jsonl, 143 frames, 5 Hz capture):

Detection Rate: 77 detections / 143 frames = 53.85%

Anchor Distribution: Shoulder (53 frames, 68.8%), Leg (22 frames, 28.6%),

None/Failed (2 frames, 2.6%)

Distance Statistics (m): Mean 3.164, Std 0.654, Min 2.658, Max

4.761 **Orientation Labels:** Facing Forward (39), Turning Camera-Right (29), Turning Camera-Left (10)

Motion Labels: Stationary (29), Approaching (30), Receding (15), Unknown (3)

Detection rate variation (43–54%) reflects pose visibility and feature density in the test environment. Shoulder anchor dominance (69–96%) indicates that the method relies primarily on upper-body geometry. The recorded MJPEG streams and JSONL logs are suitable for offline analysis and operator feedback review, with annotated video overlays enabling frame-by-frame inspection of detection quality and scale estimate transitions.

KEY RESULTS SUMMARY (275 runs, 121,042 frames)

Category	Top Finding
Tracking	LK flow used in 85.1% of frames; 17.4 ms median latency; 171 median inliers.
ORB Fallback	Used in 14.9% of frames; 70.1 ms latency; still maintains 39 inliers.
Bundle Adjustment	90.9% applied rate; 26.1% RMSE improvement; 859 ms async solve.
Map Growth	96 points/KF (median); 0.111 m translation threshold.
Relocalization	66.2% of runs hit Reloc; 82.8% recovery success rate.
Final Maps	739 points (latest); 1.45 m mean depth.

9. CONCLUSION AND FUTURE WORK

9.1 Conclusion

This thesis presented the complete design, implementation, and experimental validation of a real-time monocular VSLAM system for a PKR 60-70,000 differential-drive robot powered by a Raspberry Pi 4B. The primary research challenge—achieving robust, metric-scale indoor localization within the severe computational budget of an edge device—was met through four interlocking contributions.

First, the asynchronous pipeline architecture with a single-slot non-blocking mapping queue ensures that the 10 Hz tracking thread is never blocked by backend computation, correctly absorbing overlaps by explicitly mapping task drops rather than blocking.

Second, the custom sparse Jacobian Bundle Adjustment overcomes the massive $O(n^2)$ bottlenecks of traditional dense solvers, dropping optimization latency from blocking levels (e.g., $>4,000$ ms) down to asynchronously safe intervals (~ 400 - 1200 ms), transforming a computationally infeasible operation into a viable background task that runs concurrently with live tracking.

Third, the finite-state machine with hysteresis and breadcrumb backtracking provides an autonomous recovery mechanism from complete tracking loss, actively re-engaging the robot with the mapped region when the monocular pipeline loses structural context—a capability absent from most baseline academic monocular SLAM implementations.

Fourth, the comprehensive hardware mitigation strategy (camera exposure control, deadband compensation, EMA odometry calibration, motor pause during ORB frames) bridges the gap between ideal algorithmic assumptions and real-world actuator behavior.

Experimentally, the system maintains stable high-speed tracking (averaging 24.6 FPS over 115k logged frames) in standard office environments. It robustly executes drift correction via geometrically verified (PnP-RANSAC) Bag-of-Words loop closures powered by a highly optimized, sparse 3-DoF Pose Graph Optimization (averaging 513.6 ms). Finally, the system successfully navigates to visual target waypoints up to 4.57 meters away, yielding a mean final positional error of 17.5 cm and a highly consistent 92.1% mission success rate during structurally tracked evaluations.

These results demonstrate that low-cost camera-only autonomous indoor navigation is

achievable on commodity hardware without GPU acceleration or expensive sensor suites.

9.2 Future Work

9.2.1 Human-Robot Interaction: Person Following and Collision-Aware Navigation

The human pose estimation subsystem establishes a foundation for operator interaction beyond simple monitoring. Three research directions are feasible within current resource constraints (~2 GB free RAM, 2–3 idle Pi cores at 40% utilization):

Person Following via Face Tracking (Feasible, 3–5 Hz):

Combining face detection (YOLOFace Nano, ~20 MB model, 100–150 ms/frame on Pi) with pose-based person association enables persistent person identity across frames. A Kalman filter on face bounding-box centroids predicts position and smooths noisy detections. Navigation can then enter a FOLLOWING mode that dynamically adjusts waypoint targets to track the detected person. Overhead: <50 MB RAM, <5% CPU on a single idle core. Practical deployment: 3–5 Hz update rate (acceptable for “follow me” behavior).

Collision Avoidance with Trajectory Prediction (Feasible, Real-Time):

Estimate human velocity by differencing pose positions over a 2–3 frame window and project the human’s position 2–3 seconds ahead. If the projected position intersects the robot’s planned path, invoke Arduino hardware stop (via Pi command) or trigger a replanning event in WaypointNavigator. This mirrors the hardware collision avoidance already running on the Arduino but operates in the visual domain. Overhead: <10 MB code/state, <5 ms per frame. This adds a safety layer without requiring additional sensors.

Gaze-Aware Operator Attention (Marginal, 5–10 Hz):

A lightweight gaze estimator (ETH-XGaze, ~5 MB quantized model) on cropped face ROIs can infer where the human is looking. Use gaze direction to modulate robot behavior: if the human is looking away, increase stopping distance; if looking directly at the robot, reduce it. Overhead: ~50 ms per face per frame. Practical rate: 5–10 Hz sampling. This adds “attentiveness” awareness but is less critical than trajectory prediction.

Resource Constraints:

Dense human mesh recovery (SMPL) requires 400+ MB models and multi-threaded optimization, exceeding Pi constraints. Gesture recognition (hand pose + temporal classifiers) adds 200+ ms latency. Both are better suited for future GPU-equipped iterations or cloud offloading.

1. Robust Navigation and Active Relocalization: Currently, the navigation system follows fixed waypoints. Future work could introduce active SLAM tactics, allowing the path planner to dynamically alter the route toward previously mapped, feature-rich areas whenever tracking confidence drops. Furthermore, intelligent recovery maneuvers can be developed to methodically scan the environment to maximize the chances of intersecting with known map points for rapid relocalization instead of relying solely on blind backtracking.

2. Sensor and Hardware Enhancements: To complement the visual pipeline, the hardware platform could be upgraded with multi-directional geometric anomaly sensors. Integrating side-facing obstacle sensors, tactical bump switches for physical contact verification, and downward-facing edge/fall detectors would provide a comprehensive 360-degree safety envelope. This would allow the hardware to preemptively bypass collisions if the SLAM navigator pushes too close to constraints in unstructured environments.

3. Algorithmic Advancements via Deep Learning: While the system currently leverages rigorous classical geometry, integrating deep learning-based SLAM concepts offers robust fallbacks. Exploring neural-network-backed feature extractors, learned visual odometry, or complete hybrid data-driven SLAM approaches could bypass the inherent limitations of traditional heuristics operating in highly dynamic scenes or completely texture-starved corridors.

4. Persistent Map Storage and Relocalization: The current system rebuilds the map from scratch at every launch. Serializing the map to disk (keyframe poses, ORB descriptors, 3D points) and implementing a map-loading startup mode would enable “map-and-localize” operation—the robot localizes within a pre-built map without re-exploring the environment.

5. Full 6-DoF Pose Graph: The current PGO is simplified to 3-DoF (x, z, yaw) under the flat-ground assumption. Extending to full SE(3) Pose Graph Optimization would enable deployment on stair-climbing platforms, drones, or robots operating on uneven terrain.

REFERENCES

- [1] R. Hartley and A. Zisserman, **Multiple View Geometry in Computer Vision**, 2nd ed. Cambridge, UK: Cambridge University Press, 2003.
- [2] A. J. Davison, I. D. Reid, N. D. Molton, and O. Stasse, "MonoSLAM: Real-time single camera SLAM," **IEEE Trans. Pattern Anal. Mach. Intell.**, vol. 29, no. 6, pp. 1052–1067, Jun. 2007.
- [3] G. Klein and D. Murray, "Parallel tracking and mapping for small AR workspaces," in **Proc. IEEE/ACM Int. Symp. Mixed Augmented Reality (ISMAR)**, Nara, Japan, 2007, pp. 225–234.
- [4] R. Mur-Artal, J. M. M. Montiel, and J. D. Tardós, "ORB-SLAM: A versatile and accurate monocular SLAM system," **IEEE Trans. Robot.**, vol. 31, no. 5, pp. 1147–1163, Oct. 2015.
- [5] E. Rublee, V. Rabaud, K. Konolige, and G. Bradski, "ORB: An efficient alternative to SIFT or SURF," in **Proc. IEEE Int. Conf. Comput. Vis. (ICCV)**, Barcelona, Spain, 2011, pp. 2564–2571.
- [6] R. Mur-Artal and J. D. Tardós, "ORB-SLAM2: An open-source SLAM system for monocular, stereo and RGB-D cameras," **IEEE Trans. Robot.**, vol. 33, no. 5, pp. 1255–1262, Oct. 2017.
- [7] E. Rosten and T. Drummond, "Machine learning for high-speed corner detection," in **Proc. Eur. Conf. Comput. Vis. (ECCV)**, Graz, Austria, 2006, pp. 430–443.
- [8] M. Calonder, V. Lepetit, C. Strecha, and P. Fua, "BRIEF: Binary robust independent elementary features," in **Proc. Eur. Conf. Comput. Vis. (ECCV)**, Crete, Greece, 2010, pp. 778–792.
- [9] D. G. Lowe, "Distinctive image features from scale-invariant keypoints," **Int. J. Comput. Vis.**, vol. 60, no. 2, pp. 91–110, Nov. 2004.
- [10] B. D. Lucas and T. Kanade, "An iterative image registration technique with an application to stereo vision," in **Proc. DARPA Image Understanding Workshop**, 1981, pp. 121–130.
- [11] J.-Y. Bouguet, "Pyramidal implementation of the Lucas-Kanade feature tracker: Description of the algorithm," Intel Corporation Microprocessor Research Labs, Tech. Rep., 2001.
- [12] V. Lepetit, F. Moreno-Noguer, and P. Fua, "EPnP: An accurate $O(n)$ solution to the PnP problem," **Int. J. Comput. Vis.**, vol. 81, no. 2, pp. 155–166, Feb. 2009.
- [13] M. A. Fischler and R. C. Bolles, "Random sample consensus: A paradigm for model fitting with applications to image analysis and automated cartography," **Commun. ACM**, vol. 24, no. 6, pp. 381–395, Jun. 1981.
- [14] B. Triggs, P. F. McLauchlan, R. I. Hartley, and A. W. Fitzgibbon, "Bundle adjustment — A

modern synthesis," in **Vision Algorithms: Theory and Practice**, ser. LNCS, vol. 1883. Berlin: Springer, 2000, pp. 298–372.

[15] D. Gálvez-López and J. D. Tardós, "Bags of binary words for fast place recognition in image sequences," **IEEE Trans. Robot.**, vol. 28, no. 5, pp. 1188–1197, Oct. 2012.

[16] R. Kümmerle, G. Grisetti, H. Strasdat, K. Konolige, and W. Burgard, "g2o: A general framework for graph optimization," in **Proc. IEEE Int. Conf. Robot. Autom. (ICRA)**, Shanghai, China, 2011, pp. 3607–3613.

[17] D. Scaramuzza and F. Fraundorfer, "Visual odometry — Part I: The first 30 years and fundamentals," **IEEE Robot. Autom. Mag.**, vol. 18, no. 4, pp. 80–92, Dec. 2011.

[18] Z. Zhang, "A flexible new technique for camera calibration," **IEEE Trans. Pattern Anal. Mach. Intell.**, vol. 22, no. 11, pp. 1330–1334, Nov. 2000.
